

**Efficient instruction-level parallelism
analysis using tropical semiring ‘matrix
multiplication’**

Computer Science Tripos – Part II

May 15, 2026

Declaration

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 3450C, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: May 15, 2026.

Proforma

Candidate number: 3450C

Project title: Efficient instruction-level parallelism analysis using tropical semiring ‘matrix multiplication’

Examination: Computer Science Tripos – Part II, May 2026

Word count: 11680 ¹

Code line count: 4534 ²

Project originator: Alexandra Chadwick, *awc32*

Day-to-day supervisor: Alexandra Chadwick, *awc32*

Marking supervisor: Robert Mullins, *rdm34*

Original aims of the project

Classical techniques for calculating the upper bound of instruction level parallelism in limit studies involve sequential algorithms that restrict efficiency. This project explores a novel matrix-based technique for calculating the upper bound of instruction level parallelism in a program. The objective was to show that this approach offers better performance than traditional analysis techniques, at the cost of greater use of memory, by applying data-driven and parallel optimisations.

Work completed

The project successfully met all core objectives and explored most extensions. The extension on through-memory dependencies stated in the proposal was ultimately not considered, and the work on GPU-based compute did not yield the expected performance benefits. I successfully built several iterations of a pipeline that computed the critical paths in program execution traces, and evaluated the effect of several optimisations and strategies, both data-driven and hardware-based.

Special difficulties

None.

¹computed using `textcount`

²computed using `cloc`

List of Figures

1.1	Example dynamic dependency graph [1]	10
3.1	Partially unrolled cksum loop body in Arm Assembly, annotated with critical path dependencies	22
3.2	Histogram showing sparsity of all matrices computed while reducing the factor trace.	26
3.3	Final single-threaded pipeline.	28
3.4	Final parallel pipeline.	33
3.5	Multiplication and dependency tracking	34
3.6	Overview of the project repository and directory structure.	38
4.1	Comparing the speedup for different implementations, executed on the Small machine.	40
4.2	Comparing runtime of matrix multiplication approaches, for 64x64 matrices of varying sparsity.	41
4.3	Comparing runtime of encoding and reduction stages on different datasets, for different encoder parameters, excluding block preprocessing and loop finding.	42
4.4	Comparing compression ratio of encoder on different datasets, for different encoder parameters, excluding block preprocessing and loop finding.	43
4.5	Comparing reduction behaviour of encoder on different datasets, for representative parameters.	43
4.6	Comparing the speedup for block preprocessing and fast loop finding, using variable encoder with $r = 0.4$	44
4.7	Multi-threaded pipeline component runtime and multiplier lag time for various thread configurations.	45
4.8	Matrix multiplication throughput and overhead for various thread configurations.	46
C.1	Histogram showing sparsity of all matrices computed while reducing the cksum trace.	57
C.2	Histogram showing sparsity of all matrices computed while reducing the cksum32 trace.	58
C.3	Histogram showing sparsity of all matrices computed while reducing the factor trace.	58
C.4	Histogram showing sparsity of all matrices computed while reducing the sha1sum trace.	59

C.5	Histogram showing sparsity of all matrices computed while reducing the shuf trace.	59
C.6	Histogram showing sparsity of all matrices computed while reducing the sort trace.	60

List of Tables

2.1	Summary of trace analysis on the selected data.	18
2.2	MoSCoW analysis of project requirements and success criteria.	19
4.1	Hardware specification for each benchmark device. Boost clock was disabled for Small, so each core ran at 3.3Ghz.	39
4.2	Memory usage breakdown for each approach (MB, maximum resident set size) .	40
4.3	Breakdown of matrix multiplications across pipeline configurations, using variable encoder with $r = 0.4$. BP : Block Preprocessing, LF : Loop Finder, PE : Pair Encoding, FR : Final Tree Reduction.	44
4.4	GPU timeline breakdown for a batch of 4096 multiplications.	47

Contents

1	Introduction	10
1.1	Background: ILP and limit studies	10
1.2	Motivation	11
1.3	Contributions	11
2	Preparation	12
2.1	Understanding the matrix-based approach	12
2.1.1	The tropical semiring (max-plus algebra)	12
2.1.2	Longest paths in Directed Acyclic Graphs	13
2.1.3	Forming the columnar DDG	14
2.1.4	Alternative graph representation	15
2.2	Datasets	17
2.2.1	Data properties	17
2.3	Project design	18
2.3.1	Software engineering	18
2.3.2	Project requirements	19
2.3.3	Project tooling	19
2.3.4	Starting point	19
3	Implementation	21
3.1	Naive Baseline	21
3.1.1	Verifying the baseline	21
3.1.2	Limitations of parallelisation	23
3.2	Matrix Multiplications	23
3.2.1	A naive matrix implementation	23
3.2.2	Traversal order and cache performance	24
3.2.3	Matrix tiling	25
3.2.4	Sparse Matrices	25
3.3	Setting up the matrix approach	27
3.3.1	Final single-threaded pipeline	28

3.4	Fast expression reduction	28
3.4.1	Run-length encoding	29
3.4.2	Pair encoding	29
3.4.3	Multiple pair encoding	30
3.4.4	Block preprocessing	30
3.4.5	Variable pair encoding	31
3.4.6	Finding flat loops without frequency analysis	31
3.5	Parallelisation	33
3.5.1	Pipelined parallelism	34
3.5.2	Work-sharing	35
3.6	Accelerator-level parallelism	36
3.6.1	Tiling on the GPU	36
3.6.2	GPU Kernel	37
3.6.3	Host to device dispatch	37
3.7	Repository overview	38
4	Evaluation	39
4.1	Methodology	39
4.2	Holistic pipeline	39
4.3	Algorithmic and data-driven optimisations	40
4.3.1	Matrix multiplications	41
4.3.2	Encoder performance and compression	41
4.3.3	Single-threaded pipeline	44
4.4	Concurrency and GPU analysis	45
4.4.1	Concurrency	45
4.4.2	GPU analysis	46
5	Conclusions	48
5.1	Lessons learned	48
5.2	Future work	48
	Bibliography	49
A	Mathematical definitions and proofs	51
B	Additional code listings	52

C	Distribution of matrix sparsity in datasets	57
D	Project Proposal	61

Introduction

This project explores a novel matrix-based technique for calculating the upper bound of instruction level parallelism in a program. This project shows that this approach offers better performance than traditional analysis techniques, at the cost of greater use of memory.

1.1 Background: ILP and limit studies

Instruction level parallelism (ILP) is the simultaneous execution of independent instructions in a computer processor [12]. A pair of instructions from a thread of execution are independent when there exist no dependencies (specifically data, control or structural dependencies) between them. An instruction dependent on a previous one must be executed strictly after the former has completed execution to ensure correct execution of the entire program.

Modern superscalar processors exploit ILP to increase throughput and hide latency [15]. The absolute limit of exploitable ILP depends on the structure of the executed program. A dynamic execution of a program will have a ‘critical path’ of instructions, each data-dependent on the previous, that have to be executed sequentially.

To understand these theoretical limits, studies are conducted to measure the maximum ILP available, through analysis on dynamic execution traces. Tools are used to build a *dynamic dependency graph* (DDG), where nodes represent dynamically executed instructions, and edges represent the dependencies between them. The program’s maximum available ILP is calculated by dividing the total number of executed instructions by the length of the longest path (the critical path) through the DDG [17]. An example DDG is shown in Figure 1.1.

Execution Trace

```
load r0, A
load r1, B
r4 ← r0 + r1
load r2, C
load r3, D
r5 ← r2 + r3
r6 ← r4 + r5
store r6, S
```

Dynamic Dependency Graph

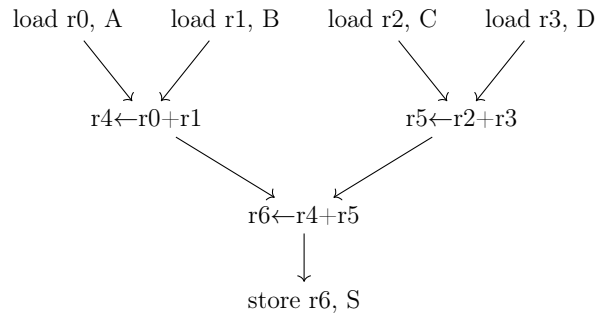


Figure 1.1: Example dynamic dependency graph [1]

1.2 Motivation

Traditionally, limit studies compute this longest path using a single-pass sequential sweep over the execution trace. [17, 14, 10] At each step they calculate the length of dependency paths ending at each register, and pick out the longest path at the end of the trace. Because the algorithm traverses the trace sequentially to propagate dependencies, it cannot be heavily parallelised, nor can it exploit the memoisation of program structures such as loops. This sequentiality limits the ability to handle realistic execution traces containing trillions of instructions, which is important for capturing the effects of long-term dependencies found in complex, real-world software. Furthermore, this analysis is rarely a single-pass operation. Limit studies evaluate programs across comprehensive benchmark suites under various parameters, such as restricted instruction windows or different branch prediction models [17, 4, 14].

To break this sequential bottleneck, Chadwick [3] proposes a matrix-based approach. By representing instruction dependencies as matrices, and using matrix multiplication in the max-tropical semiring, computing the longest path in the DDG becomes a problem of reducing a large expression of matrix multiplications. Because matrix multiplication is associative, the expression can be evaluated in any order, and intermediate results fully encode the dependency information of the instructions they range over. This enables a range of optimisations when computing ILP limits, including thread-parallelisation of computations and memoisation of repetitive intermediate results.

One additional benefit of the associativity of the matrix expression is that it trivialises instruction window scaling. Consider three windows of instructions: A, B, and C (which is comprised of windows A and B consecutively). To compute ILP information for each window, the traditional approach must be executed 3 times, once for each window. However, the matrix-based approach will only need to run twice, on A and B, and the dependency information matrix for C can simply be obtained by multiplying the result matrices from A and B.

1.3 Contributions

In this project, I explore several approaches for offline computation of the upper limit of ILP in a program, and compare their performance. The first is a sequential approach mirroring the classical trace-analysis techniques, based on the PerPI tool [10]. We will refer to this as the ‘naive baseline’ throughout the project. I then explore Chadwick’s matrix-based approach, implementing a series of algorithmic and hardware-level optimisations. Specifically, I use sparse matrix representations to reduce memory overhead, and develop data-level heuristics to determine optimal expression evaluation orders. Finally, I introduce parallel optimisations, leveraging multi-threading and accelerator-level parallelism to distribute the computational workload.

I show experimentally in Section 4.2 that a single-threaded pipeline equipped with the data-driven optimisations beats the performance of the naive baseline in most cases. Furthermore, by distributing expression reduction and matrix multiplication workloads over multiple cores, I show that the parallel approach is at least twice as fast as the baseline, demonstrating the scalability of the matrix-based approach in analysing large execution traces.

Preparation

This chapter outlines the work done in the setup of the project. Section 2.1 explains the theoretical background of the matrix-based approach. Section 2.2 explores the data on which the analysis tool is executed. Finally, Section 2.3 outlines the software engineering methodologies used, project requirements, and chosen tooling.

2.1 Understanding the matrix-based approach

The applications of tropical semirings to dynamic dependency graphs, particularly the sections on register-only longest chains (2.1.3) and alternative graph structures (2.1.4) are closely adapted from the methodologies established by Chadwick [3].

2.1.1 The tropical semiring (max-plus algebra)

Tropical analysis is the study of the tropical semiring. There are two variants of the tropical semiring, namely the min-plus and max-plus algebras. We apply the max-plus convention to the problem of *longest* paths in directed acyclic graphs.

Definition 2.1.1 (Tropical semiring). The tropical semiring [18], or max-plus algebra, is the object $(\mathbb{R} \cup \{-\infty\}, \oplus, \odot)$ with operations defined as:

$$\begin{aligned} a \oplus b &= \max(a, b) \\ a \odot b &= a + b \end{aligned}$$

The tropical semiring satisfies the axioms of a semiring (defined in Appendix A). It is not a ring as the max operator does not have an inverse.

Definition 2.1.2 (Matrix operations in the tropical semiring). For any $n \times n$ matrices $X = (x_{ij})$ and $Y = (y_{ij})$, the entries of $U = X \oplus Y$ and $V = X \odot Y$ are calculated as [13]

$$u_{ij} = x_{ij} \oplus y_{ij}, \text{ and } v_{ij} = \bigoplus_{k=1}^n x_{ik} \odot y_{kj}$$

The matrix operations $\oplus$ and $\odot$ are associative owing to the commutativity and associativity of the underlying addition operator, and the distributivity of the multiplication operator over addition.

The Strassen algorithm [16] is a matrix multiplication algorithm with a better asymptotic complexity ($O(n^{\log_2 7})$) than the above definition ($O(n^3)$). However, using Strassen's algorithm requires an additive inverse, which does not exist in the tropical semiring (as max has no inverse), and so it is not applicable to our particular use-case.

2.1.2 Longest paths in Directed Acyclic Graphs

As established in Section 1.1, finding the length of the critical path in a program requires computing the longest path in its DDG. One algorithm [3] to find the longest path between nodes A and B in a graph is to recursively take the maximum of all path lengths from all neighbours of A to B , plus the cost of the edge from A to the chosen neighbour. We can see that this amounts to a Bellman recursion equation [2] involving only the max and plus operators:

$$v(A, B) = \begin{cases} 0 & \text{if } A = B \\ \max_{\forall C. A \rightarrow C} \{ \text{weight}(A \rightarrow C) + v(C, B) \}, & \text{otherwise} \end{cases}$$

We observe that if we have an adjacency matrix (where the cells in that matrix represent the cost of the edge between the pair of nodes in that row/column, and $-\infty$ if there is no edge), performing tropical semiring matrix multiplication of this matrix with itself corresponds to one step of our longest-path algorithm.

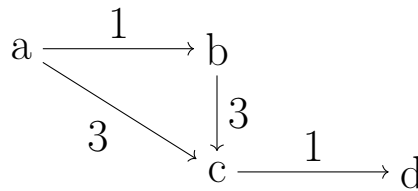
Formally, we consider an adjacency matrix $X^2 = X \odot X$, and denote its entries by $x_{ij}^{(2)}$. We have that $x_{ij}^{(2)} = p$ if and only if there exists at least one path of length p from node i to node j in the corresponding graph, consisting of at most two edges, and that path is the longest such path. This extends to any integer exponent q [13].

Theorem 2.1.1 (Matrix exponentiation results in longest paths). For any integer $q > 0$, the matrix X^q has the entry $x_{ij}^{(q)} = p$ only if there is a path of length p with at most q edges from i to j , and that path is the longest such path.

Proof Sketch. The proof proceeds by induction on q , following the steps of the previously defined Bellman recursion equation. See Appendix A for the full proof. $\square$

Therefore an adjacency matrix raised to the power infinity will converge to the longest distance matrix for that whole graph.

For example, consider the graph:



The corresponding adjacency matrix would be:

$$X = \begin{pmatrix} 0 & 1 & 3 & -\infty \\ -\infty & 0 & 3 & -\infty \\ -\infty & -\infty & 0 & 1 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

So, in the tropical semiring:

$$X^2 = \begin{pmatrix} 0 & 1 & 4 & 4 \\ -\infty & 0 & 3 & 4 \\ -\infty & -\infty & 0 & 1 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}, \quad X^3 = \begin{pmatrix} 0 & 1 & 4 & 5 \\ -\infty & 0 & 3 & 4 \\ -\infty & -\infty & 0 & 1 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

$$= X^4 = X^5 = \dots = X^\infty$$

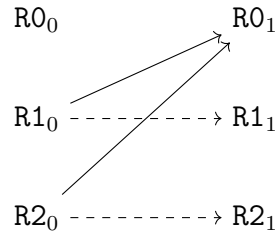
This corresponds to the longest distance matrix between all pairs of nodes.

2.1.3 Forming the columnar DDG

As defined in Section 1.1, a standard DDG represents each dynamically executed instruction as a single node. While this formulation is intuitive for visualising the program flow, the structure of the graph is arbitrary, and we would wish to impose a more rigid structure to allow for future optimisations, as outlined in Section 2.1.4. To do this, we can encode the transformation of the machine's dependency state from one cycle to the next. We introduce a new expanded graph structure, which has one node per register per dynamic instruction.

Let us first consider the dependencies for instructions only affecting registers. Since each instruction has a fixed set of registers which it accesses, it has a fixed dependency structure, and so each instruction in the program trace will have a fixed subgraph in the expanded DDG encoding the program's dependency information.

For example, we could encode the dependencies for the instruction `R0 = ADD R1, R2` as the subgraph:



dashed edges are 0 cost

The instruction reads from R1 and R2 and uses these values to produce a new value stored in R0. On the left of the subgraph is the register state before the instruction execution; on the right is after. The characteristics of the subgraph are as follows:

- Since the value $R0_1$ is dependent on $R1_0$ and $R2_0$, there are cost-1 edges in the graph from $R1_0$ and $R2_0$ to $R0_1$, extending the dependency chain from the source registers to the destination by 1 cycle.
- The values in R1 and R2 are unchanged by the instruction, so their respective states have 0-cost edges between them. This allows dependency chains from the source registers to ‘skip over’ this particular instruction that does not affect them.
- Finally, since the value in R0 is modified, there is no 0-cost edge between the respective states, as the old value is discarded and a new chain starts at the new value.

The full DDG will consist of several of these subgraphs, horizontally glued together, with one new set of dependency edges for each additional instruction.

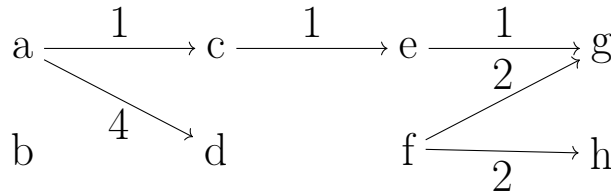
The true critical path often involves memory operations, as a value may be copied to memory from a register. Then, potentially millions of instructions later, that value is read into a register and used, contributing to a data dependency. We have not accounted for this here. While Chadwick [3] has suggested approaches to track through-memory dependencies, this project does not do so, as they require dealing with much larger matrices or significantly complicating the structure of the datasets, which fell outside of the project’s scope.

2.1.4 Alternative graph representation

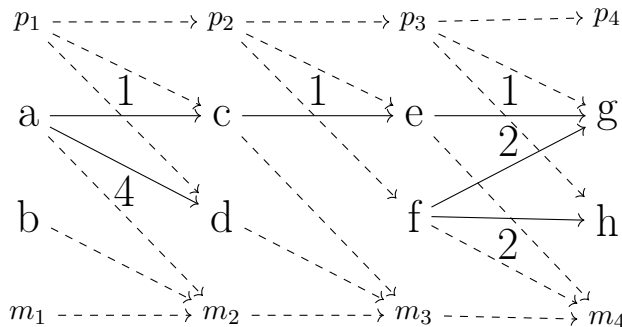
Constructing a full adjacency matrix for the DDG structure in 2.1.3 is computationally unfeasible for realistic program traces, as they contain trillions of dynamic instructions.

Chadwick [3] proposes an alternative representation to the full adjacency matrix. There is a particular structure in the DDGs we construct: they are more long than wide. In the DDG, edges *only* exist between adjacent columns, and so a full adjacency matrix X is mostly filled with $-\infty$. We can instead model a DDG as a sequence of adjacent columns.

Consider an arbitrary graph with the property of being more long than wide:



To utilize column-wise matrices for longest-path calculations, the graph must first be padded. We introduce a set of ‘dummy’ nodes to the first and last columns, connected by 0-cost edges. This guarantees that any local path within the interior of the graph is extended to touch both the initial and final columns.



dashed edges are 0 cost

Now we can represent the graph with one matrix per adjacent-column pair, encoding the connectivity between those two columns. The connectivity between the first two columns is represented as:

$$X_{1,2} = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ -\infty & 1 & 4 & 0 \\ -\infty & -\infty & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

where the m th row and n th column of this matrix represent the connection from the m th node in the first column of the graph to the n th node in the second column of the graph.

The remaining two column pairs are represented as:

$$X_{2,3} = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ -\infty & 1 & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}, \quad X_{3,4} = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ -\infty & 1 & -\infty & 0 \\ -\infty & 2 & 2 & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

These matrices are each 4×4 whereas a full adjacency matrix for the original graph would be 7×7 . We can see that the column-wise matrices will use less storage than the adjacency matrix (48 integers vs 49 integers), and this alternative becomes increasingly favourable as the graph gets longer.

In the tropical semiring, computing the matrix product

$$X_{3,4} \odot X_{2,3} \odot X_{1,2} = \begin{pmatrix} 0 & 2 & 2 & 1 \\ -\infty & 3 & -\infty & 4 \\ -\infty & -\infty & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

yields a matrix representing the longest path from the m th node in the first column to the n th node in the final column of the graph. Because we have ensured every path in the graph connects to the first and last column, the largest individual value in this matrix is the length of the longest path in the graph, namely the path of length 4 connecting a in the first column to m_4 in the final column, via d and m_3 .

The correctness of the column-wise approach for calculating longest paths in graphs is a corollary that follows directly from Theorem 2.1.1.

Corollary 2.1.1 (Correctness of the column-wise formulation). In a standard adjacency matrix X , computing X^q checks every possible intermediate node k in the entire graph to find paths of length q , as shown by Theorem 2.1.1.

In a column-wise representation of the DDG, when we multiply adjacent column matrices (e.g., $X_{2,3} \odot X_{1,2}$), we are simply performing the exact operations for finding the longest path between columns 1 and 3, but restricted only to the intermediate nodes k located in column 2. This is valid as edges in the DDG only exist between adjacent columns. Because the padding of 0 cost edges ensures every path spans from the first column to the final column, multiplying all consecutive column matrices sequentially computes the maximum path length across the entire graph.

We can use the column-wise approach to represent the alternative DDG structure from Section 2.1.3. Each dynamic instruction has a static set of dependencies represented by a matrix, and there are many more dynamic instructions in the DDG than the number of static instructions in

the program. Therefore, there will be frequent reuse of matrices in the equation, which further reduces the actual memory required to store the DDG.

This defines the final shape of the data that we will be working on as a mathematical expression consisting of a series of matrix multiplications. The corollary theoretically satisfies the success criteria for the MPF requirement in Table 2.2.

2.2 Datasets

I selected execution traces from several GNU coreutils programs (v8.32) [8] to benchmark the analysis tool, all executed on an AArch64 architecture. The execution traces are given as a list of branches, which can be interpreted as a series of basic blocks that are executed at runtime. A basic block is a chain of instructions executed sequentially without a branch.

The chosen program executions included `cksum` (calculating the CRC of 1MiB and 32MiB files of random bytes), `factor` (factorising a 22-digit integer), `sha1sum` (computing the SHA1 message digest of a 1MiB file of random bytes), `sort`, and `shuf` (respectively sorting and shuffling text files of random hexadecimal digits). The source data can be found at <https://www.cl.cam.ac.uk/~awc32/cam-only/tropical-ilp-p2p/>. The traces were collected by Alexandra Chadwick (*awc32*) using custom tooling.

Most standard benchmark suites consist of huge execution traces, and smaller suites only contain trivially tiny traces. The chosen traces achieve a middle ground of being small enough for quick testing during iterative development, but are also real, commonly used programs that are doing realistic work. The traces represent a range of workloads, from highly repetitive loop workloads in `cksum` and `sha1sum`, to more complex branching in `factor` and `sort`.

2.2.1 Data properties

Many of the optimisations in this project revolve around exploiting patterns in the datasets by varying evaluation orders and memoising intermediate results. To motivate these optimisations, we can perform an analysis on the execution traces to quantify the extent of these data patterns.

We introduce terminology for distinguishing various types of repeating patterns [19]. Repeating substrings (or *repeats*) in a string can be *tandem* repeats, where successive repeat substrings immediately follow each other, without overlapping. They characterise loops in a program trace. A tandem repeat is *primitive* if it is itself not composed of tandem repeats of another substring. A set of tandem repeats is *maximal* if there is no identical substring immediately preceding or succeeding the sequence of tandem repeats. We refer to primitive and maximal tandem repeats as *PM repeats*, and throughout the project, ‘loops’ will refer to PM-repeats in the execution trace.

Table 2.1 summarises a trace analysis on the selected datasets, carried out with a tool based on the loop finder in Section 3.4.6. We define **Period** as the length of the repeated substring in basic blocks, **Average Consecutive Iterations (ACI)** as the mean number of times the period of a pattern repeats without interruption, and **Total Iterations (TI)** as the total number of times the period of a pattern exists in the program trace. **ACI** and **TI** correspond to the length of a PM-repeat and the total length of all PM-repeats with the same repeating substring respectively.

Dataset	Instructions (Total)	Basic Blocks (Total)	Biggest Pattern			
			Period	ACI	TI	Coverage
cksum	7526926	1068940	1	65535	1048560	98.1%
cksum32	235132057	33580788	1	65535	33553920	99.9%
factor	220632540	18074022	5	20	993556	27.5%
sha1sum	16994993	91070	1	3	49155	54.0%
sort	10943690	1216990	17	2	7536	10.5%
shuf	16361747	2147424	30	3	16007	22.3%

Table 2.1: Summary of trace analysis on the selected data.

The table shows that all datasets exhibit a ‘largest pattern’ covering a significant proportion of the trace. This pattern is built of several PM-repeats with the same repeating substring, but separated/split by other symbols. We can deduce that each of the PM-repeats correspond to an invocation of the same loop, which executes for some varying number of iterations. Each invocation of the loop is separated by other instructions, with their own varying dependency information.

This type of pattern can be exploited in two ways:

1. Consecutive repetitions of matrices can be quickly reduced down into a single matrix using a fast exponentiation algorithm.
2. The matrices representing instructions in one iteration of the loop can be multiplied together to obtain a singular matrix representing the dependency information for that single iteration. All other iterations of that loop body can then be replaced with the singular result matrix without any extra computation.

2.3 Project design

This section outlines the software engineering methodology adopted during implementation, specification of the project requirements, and technical justification for the chosen tools.

2.3.1 Software engineering

I adopted the iterative spiral methodology for this project. Given the experimental nature of performance optimisation, this methodology facilitated continuous cycles of planning, implementation, and informal profiling to identify performance bottlenecks, informing the next iteration cycle.

To maintain codebase integrity during high-frequency iterations, code was regularly committed and pushed to GitHub, using feature branches to implement large changes. Commits from branches were squashed upon merge to provide a clean, linear history of major architectural milestones.

2.3.2 Project requirements

Requirements are listed in Table 2.2 and categorised by their priority, according to the MoSCoW method guidelines [5].

ID	Category	Requirement	Success Criteria
NB	<i>Must-have</i>	Naive Baseline Implementation	A valid reference implementation to provide a baseline for performance comparisons.
MPF	<i>Must-have</i>	Matrix-Based Path Finding	Correctness verification: ensure the matrix approach yields identical results to the naive baseline.
PE	<i>Must-have</i>	Pattern Exploitation	Demonstrable reduction in redundant computations by identifying and optimising common patterns in the data.
MP	<i>Should-have</i>	Multithreaded Parallelism	Achievement of performance scaling on multi-core CPU architectures using an asynchronous sub-problem approach.
GA	<i>Could-have</i>	GPU Acceleration	Successful offloading of matrix multiplications to an accelerator, identifying the point at which throughput overtakes overhead.
MT	<i>Won't-have</i>	Through-memory dependency tracking	Excluded from scope to focus on local hardware optimisations.

Table 2.2: MoSCoW analysis of project requirements and success criteria.

2.3.3 Project tooling

The choice of tools was driven by the need for low-level hardware transparency, balanced with architectural flexibility. C++ was selected as the primary programming language as it allows for fine-grained memory control (which is important for performance-sensitive code for avoiding expensive garbage collection), and provides modern object-oriented paradigms enabling the use of design patterns to easily swap between strategies when testing and benchmarking.

While CUDA provides a highly optimized environment for Nvidia hardware, OpenCL was chosen for its cross-platform portability. This allowed for development and kernel debugging on integrated Intel graphics (laptop) before deploying to a high-performance discrete Nvidia GPU for final benchmarking.

2.3.4 Starting point

I had not used C++ or OpenCL previously. While I had prior experience programming in C, there were still many concepts I had to learn through internet resources or discussion with my

supervisor. These included the C++ templates and concepts features, multi-threading, and GPGPU programming through OpenCL.

Part IA/IB courses provided general knowledge used when debugging issues, familiarity with the theory behind the matrix approach, and information on the OpenCL programming model. Part II courses and modules, most notably Advanced Computer Architecture and Multicore Semantics & Programming respectively solidified my understanding of ILP and limit studies, and of multi-threaded programming.

I used the datasets listed in Section 2.2 alongside custom tooling provided by Alexandra Chadwick to generate information for the dependency matrices and branch structure of the trace.

No code was written before the project proposal deadline of 10/10/2025.

Implementation

This chapter describes the implementation of the different components of the analysis tool, working from the naive baseline (Section 3.1), exploring matrix representations and computations (Section 3.2), the basics behind matrix expression reduction (Section 3.3), continuing to data-driven optimisations (Section 3.4), and concluding with acceleration of compute using additional hardware (Sections 3.5 and 3.6).

3.1 Naive Baseline

As described in Section 1.2, classical techniques for ILP limit studies rely on sequential algorithms that iterate through a program trace. While public tools such as PerPI [10] employ these sequential algorithms, they are not suitable as a baseline in this project. PerPI is an online analysis tool that is injected into x86 binaries using Intel’s Pin tool, and computes ILP at runtime. This conflicts with Chadwick’s approach, which performs an offline analysis on AArch64 execution traces dumped from regular execution, making parametrisation of studies easier. To enable a more compatible comparison, I implemented a custom baseline, inspired by PerPI’s core procedure for computing ILP. Algorithm 1 calculates the longest path through the DDG formulated in Section 2.1.3.

For each instruction in the program trace, the algorithm

1. Evaluates static read dependencies to find the maximum dependency chain length ending in a given register.
2. Evaluates static write dependencies to propagate this longest chain forwards.

3.1.1 Verifying the baseline

We can verify a result from the algorithm by inspecting a small example.

The `cksum` program trace (see Section 2.2.1) has a small inner loop of 7 instructions; this basic block repeats 65536 times per invocation.

```

1 struct StaticDep {
2     int[] reads;
3     int[] writes;
4 }
5
6 function findLongestPath(int[] trace, map<int, StaticDep> deps) -> int {
7     int[] curDeps = [0,...,0];
8
9     int maxChainLength = 0;
10
11    for (int instr : trace) {
12        int chainLength = 1;
13
14        /* Examine read dependencies */
15        for (int reg : deps[instr].reads) {
16            chainLength = max(chainLength, curDeps[reg] + 1);
17        }
18
19        maxChainLength = max(maxChainLength, chainLength);
20
21        /* Propagate write dependencies */
22        for (int reg : deps[instr].writes) {
23            curDeps[reg] = chainLength;
24        }
25    }
26
27    return maxChainLength;
28 }

```

Algorithm 1: Sequential dependency chain tracking

```

...
1c68 eor x19, x0, x19, lsl #8
1c6c cmp x2, x3
1c70 b.ne 1c58
1c58 ldrb w1, [w2], #1
1c5c eor x1, x1, x19, lsr #24
1c60 and x1, x1, #0xff
1c64 ldr x0, [x23, x1, lsl #3]
1c68 eor x19, x0, x19, lsl #8
1c6c cmp x2, x3
1c70 b.ne 1c58
1c58 ldrb w1, [w2], #1
1c60 eor x1, x1, x19, lsr #24
...

```

The diagram illustrates the critical path dependencies in the Arm assembly code. Red arrows indicate the flow of data dependencies between registers across iterations. The critical path starts at the `ldr x0, [x23, x1, lsl #3]` instruction, which depends on `x1` from the previous iteration. This is followed by `eor x19, x0, x19, lsl #8`, which depends on `x0` from the previous iteration. The path then continues through `cmp x2, x3` and `b.ne 1c58` to the `ldr x0, [x23, x1, lsl #3]` instruction in the next iteration. The path also includes `ldr x0, [x23, x1, lsl #3]` and `eor x19, x0, x19, lsl #8` from the previous iteration, showing how the critical path repeats and extends across loop iterations.

Figure 3.1: Partially unrolled cksum loop body in Arm Assembly, annotated with critical path dependencies

Figure 3.1 shows the Arm assembly code for cksum’s inner loop, including instructions from previous and successive loop iterations. Data dependencies from written registers to read registers are drawn on the figure in red, showing the critical path for one iteration of the loop. Note that the critical path repeats and extends across loop iterations.

The loop body contains 7 instructions, with a critical path length of 4, yielding an expected

ILP limit of $7/4 = 1.75$.

Executing the sequential algorithm over one invocation of the inner loop (65536 iterations) processes a total of 458752 instructions, and finds a critical path length of 262145. This path length includes the outgoing edge from the last execution of instruction 1c68, so 1 is subtracted. Then, the ILP limit is $458752/262144 = 1.75$, verifying our baseline. This satisfies the success criteria for the NB project requirement from Table 2.2.

3.1.2 Limitations of parallelisation

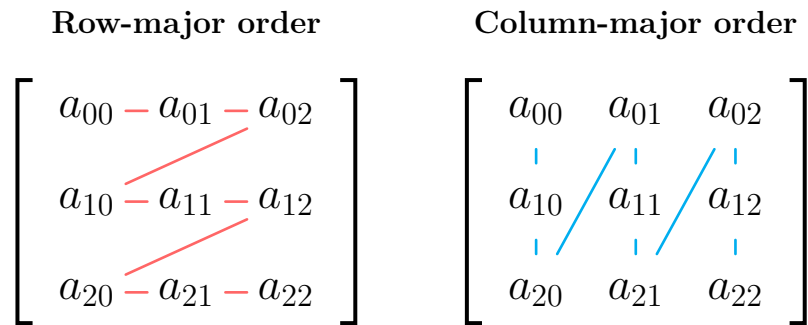
Because Algorithm 1 only stores the longest chain ending at each register and discards all others, the `curDeps` array cannot memoise intermediate results for small instruction windows. Instead, all the longest dependency chains starting at some register and ending at another have to be encoded in the result, which pushes us towards the matrix representation. Propagation of dependencies for all chains is simply the multiplication of the matrices.

3.2 Matrix Multiplications

As introduced in Section 2.1, this section details several approaches to representing and multiplying matrices together in the tropical semiring on the CPU. The performance of these approaches is evaluated in Section 4.3.1.

3.2.1 A naive matrix implementation

Matrices are typically represented by a 2-dimensional array of elements. Row-major order and column-major order are methods for storing multidimensional arrays in linear memory (arrays of a single dimension).



This implementation chooses row-major as the data representation for our matrices. The index of the element $a_{i,j}$ of an $n \times m$ matrix A in the row-major array representing the matrix is $im + j$.

We can implement basic matrix multiplication (Algorithm 2) using the formula from Definition 2.1.2.

Modern CPU caching exploits spatial locality of reference, where if a piece of data is accessed by the CPU, other pieces of data in close proximity in memory will be in the same cache line or prefetched into the cache, therefore reducing the chance of an expensive cache miss

```

1 function multiply(Matrix<n,m> X, Matrix<m,p> Y) {
2   Matrix<n,p> V;
3
4   for (i : [0,...,n-1]) {
5     for (j : [0,...,p-1]) {
6       int sum = -∞;
7       for (k : [0,...,m-1]) {
8         sum = max(sum, X[i,k] + Y[k,j]);
9       }
10      V[i,j] = sum;
11    }
12  }
13
14  return V;
15 }

```

Algorithm 2: Basic matrix multiplication algorithm.

on subsequent cycles. In the chosen algorithm, matrices X and Y are both stored in row-major order. However, inside the inner loop, X is being traversed row-wise, and Y is being traversed column-wise. The elements of Y being accessed on consecutive iterations are not in close proximity to each other in memory, and so cache misses dominate the running time, rather than the actual calculations.

This issue can be avoided by storing matrices in both row-major and column-major order. Then, depending on whether the matrix is the right or left operand in a multiplication, we can select the most cache-friendly option to traverse. This however, doubles our memory usage and so, we instead try to change the order in which we access elements of the matrices.

3.2.2 Traversal order and cache performance

The three loops in the previous matrix multiplication can be arbitrarily swapped with each other without an effect on correctness or asymptotic running time. We can swap our k and j loops so that the inner loop traverses Y in row-major order, resulting in Algorithm 3

```

1 function multiply(Matrix<n,m> X, Matrix<m,p> Y) {
2   Matrix<n,p> V;
3
4   for (i : [0,...,n-1]) {
5     for (k : [0,...,m-1]) {
6       int xVal = X[i,k];
7       if (xVal == -∞) continue;
8
9       for (j : [0,...,p-1]) {
10        V[i,j] = max(V[i,j], xVal + Y[k,j]);
11      }
12    }
13  }
14
15  return V;
16 }

```

Algorithm 3: Cache-aware matrix multiplication algorithm

Apart from the improved cache performance, there are three other optimisations.

Value hoisting Since the indices i and k do not change in the inner loop of the algorithm, we can hoist the retrieval of $X_{i,k}$ outside of the loop.

Sparsity This optimisation concerns line 7 of the algorithm. To make sense of it, we can consider an extension to the previous matrix multiplication definition:

$$v_i = \bigoplus_{k=1}^n x_{ik} \odot y_k.$$

This demonstrates that the i th row of V is a linear combination of the k rows of Y . Crucially, if $x_{ik} = -\infty$, the corresponding row Y_k is scaled to the additive identity, contributing nothing to the final sum. Thus, we can skip the dot product for any row Y_k where $x_{ik} = -\infty$.

Vectorisation Finally, since Y is traversed contiguously in the inner loop, this makes it easily open to optimisations using vector instructions. Vector instructions, also known as SIMD (Single-Instruction, Multiple-Data) [7], perform the same operation on several operands, exploiting data-level parallelism. The max and addition operations can then be performed on the adjacent data from several iterations of the inner loop using vectorised variants, increasing the number of elements processed per iteration.

3.2.3 Matrix tiling

One other common method of improving the cache performance of matrix multiplication algorithms is *tiling*. Tiling implicitly divides the matrix into smaller square blocks, computing one tile at a time to keep the active working set within the cache. This improves the spatial locality of memory accesses, as elements at the beginning of a row are not evicted by the time we get to the end of the row, which is a problem for large matrices.

While matrix tiling is a popular optimisation, it introduces complex control-flow overhead. Because we only track register dependencies, our matrices are only approximately 64×64 in size. Assuming cells hold 32-bit integers, an entire matrix requires just 16384 bytes, fitting comfortably within the 32KiB or 64KiB L1 data cache of a modern processor core. Therefore, tiling is unnecessary in our case and the control flow overhead is wasteful; instead, the contiguous linear traversal already provides the improved performance by aligning perfectly with hardware prefetchers.

3.2.4 Sparse Matrices

In Section 3.2.2, we presented an optimisation that exploited the sparsity of matrices. This provided a significant speedup to multiplications, and so we may be able to extract further benefits from exploiting matrix sparsity.

First, we can analyse the sparsity of all matrices encountered during the expression reduction.

Figure 3.2 shows that a significant proportion of the matrices we encounter are composed of negative infinity values. Omitting these values from storage significantly reduces memory consumption.

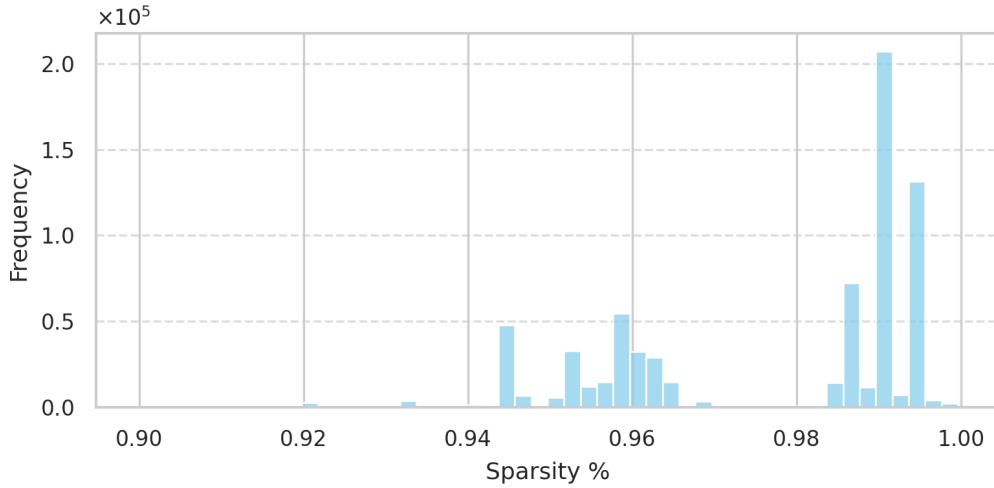


Figure 3.2: Histogram showing sparsity of all matrices computed while reducing the factor trace.

CSR Matrix Format

There exist many formats for sparse matrix representation. We selected CSR (Compressed Sparse Row) format, as it is also used in the chosen multiplication algorithm presented by Gustavson [11].

Because we operate within the tropical semiring, our implementation stores only non-negative-infinity values rather than the traditional non-zero values. We quantify matrix ‘sparsity’ with the *nni* count, the number of values in a matrix that are not $-\infty$.

The CSR representation for a $m \times n$ matrix A is given by three one-dimensional arrays. Two arrays list column indices (JA) and numerical values (VA) of the *non-negative-infinity* matrix entries. The arrangement of these elements is row-wise. The third array is a set of row address pointers (IA) where the i th element of IA is the address in both JA and VA of the first *non-negative-infinity* element of the i th row of A [11].

For example, the matrix below is stored as the following arrays:

$$\begin{pmatrix} 5 & -\infty & -\infty & -\infty \\ -\infty & 8 & -\infty & -\infty \\ -\infty & 1 & 3 & -\infty \\ -\infty & 6 & -\infty & -\infty \end{pmatrix} \rightarrow \begin{array}{l} IA = [0, 1, 2, 4, 5] \\ JA = [0, 1, 1, 2, 1] \\ VA = [5, 8, 1, 3, 6] \end{array}$$

To extract a row i , we define:

```
rowStart = IA[i]
rowEnd   = IA[i + 1]
```

Then we take slices from JA and VA starting at `rowStart` and ending just before `rowEnd`.

In this case the CSR representation contains 15 entries, compared to 16 in the original dense representation. The CSR format saves on memory only when $nni < (m(n-1) - 1)/2$.

Sparse Matrix Multiplication

Gustavson [11] proposes an algorithm for multiplying two matrices A and B in CSR format. Again, as B is stored in row-wise it is more efficient to traverse it similarly, and so the algorithm uses the linear combination formulation presented in Section 3.2.2. To find the i th row of C , for each non-negative-infinity value A_{ik} , scale the row B_k by A_{ik} , and add it to the current row of C .

The challenge for the sparse representation is summing these sparse rows together efficiently. To do this, Gustavson uses a dense accumulator and a ‘switch’ array. The dense accumulator $\mathbf{x}$ is an array used to sum up values for the current row. The switch array $\mathbf{xb}$ is an array used to track which columns have been touched in the current row calculation.

The accumulation of C ’s rows is done inside the innermost loop as follows:

- If $\mathbf{xb}[k] == i$, we have already seen this column for the current row. Simply add the value to the accumulator $\mathbf{x}$.
- Otherwise, this is the first time seeing this column for the current row. Initialize $\mathbf{x}$, and update $\mathbf{xb}$ to i .

After the accumulation is complete, the algorithm copies the active columns and their associated values into C . The algorithm requires dynamic allocation for C ’s structures, as the sparsity of C is not known until the multiplication is carried out.

The pseudocode for sparse matrix multiplication is given in the appendices (Algorithm 5).

3.3 Setting up the matrix approach

Having explored matrix multiplications in the previous section, we need an efficient representation and reduction strategy for the entire expression for the implementation’s core. This section details memory management, deterministic matrix identification, and a basic reduction strategy.

The ‘matrix expression’ is formed using pointers to the matrix objects on the heap. Because a matrix holding the static dependency information of an instruction may be referenced multiple times in the program trace, we employ a reference counting mechanism and a pool of ‘free’ and ‘used’ matrices.

Initialisation: When a matrix is first created, its reference count is set to one. The memory for the matrix is pulled from a pool of existing free matrix objects, or is newly allocated if no memory is available.

Expression building: Each time a pointer to a matrix is added to the expression, its reference count is incremented.

Expression reduction: Each time a pointer to a matrix is removed from the expression, its reference count is decremented. If the count reaches zero, the matrix’s memory is marked as free and waits to be reused.

This approach ensures that static matrices, which may appear millions of times in a long execution trace, are only stored once in memory.

The identity of two matrices in the expression can be quickly checked by comparing their pointers. However, while this is efficient for identity checks when memoising, it can introduce non-determinism. In Section 3.4.2, matrix pointers are used as a tie breaker when comparing the frequency of pairs. Due to the non-determinism of the OS memory allocator and Address Space Layout Randomisation (ASLR), pointer-based tie-breaks result in different evaluation orders across separate runs. To guarantee reproducibility, each matrix is assigned a monotonically increasing ID upon its creation, and these unique IDs are compared to tie-break.

To avoid redundant computations, we implement a memoisation table. This table maps a pair of matrix pointers P_A, P_B to a result matrix pointer P_C , representing the operation $A \odot B = C$. Before performing a multiplication, the system queries this map. If a hit is found, the reference count of P_C is incremented, and the multiplication operation skipped.

To reduce the expression to the final matrix without exploiting data patterns, we can use a multi-pass tree-reduction approach. At each pass, adjacent matrices are multiplied together and this is repeated until only one matrix is left. This allows for multiplications to be batched, which will be useful when delegating computation (see Section 3.5.1).

3.3.1 Final single-threaded pipeline

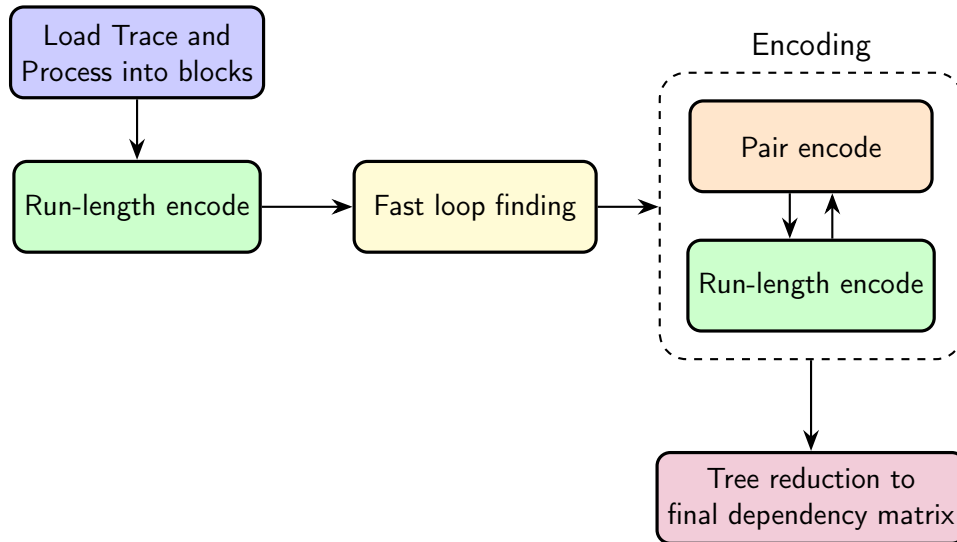


Figure 3.3: Final single-threaded pipeline.

Figure 3.3 provides an overview for the single threaded pipeline for efficiently reducing large matrix expressions. Block trace processing is discussed in Section 3.4.4, run-length encoding in Section 3.4.1, fast loop finding in Section 3.4.6, and pair encoding in Section 3.4.2.

3.4 Fast expression reduction

This section describes several data-driven optimisations for reducing the work required to reduce the matrix expression, by exploiting patterns in the data, which were explored in

Section 2.2.

3.4.1 Run-length encoding

Fast exponentiation is a method that allows fast computation of large positive integer powers of a square matrix. It is based on the recursive equation:

$$A^n = \begin{cases} A & \text{if } n = 1 \\ A \odot (A^{(n-1)/2})^2 & \text{if } n > 1 \text{ is odd} \\ (A^{n/2})^2 & \text{if } n \text{ is even} \end{cases}$$

This reduces the number of matrix multiplications required to evaluate A^n from $O(n)$ to $O(\log n)$. Fast exponentiation is used in a run-length-encoding pass that iterates over the matrix expression and collapses all repeated symbols.

3.4.2 Pair encoding

The next optimisation aims to maximise the effectiveness of a single matrix multiplication by exploiting its frequency in the broader expression. By memoising the result of the most frequent adjacent matrix pair, we can replace all instances of the pair throughout the expression. This maximises reduction while minimising computational effort.

This results in a multi-pass algorithm. During each iteration, the algorithm scans the expression to find the most frequent adjacent pair of matrices. It then computes the result of their multiplication, and replaces all instances of the pair with the result matrix. This process is repeated until the maximum frequency of a pair reaches a specified cutoff threshold.

This approach directly mirrors dictionary-based compression algorithms, specifically Byte Pair Encoding (BPE) [9]. BPE iteratively identifies and substitutes the most frequently occurring adjacent symbols in a sequence to minimise data size. This reflects the core principle of entropy coding, which achieves an optimal code by assigning the smallest representational cost (shortest codes) to the most frequent occurrences.

As a consequence, this algorithm is able to effectively exploit the loop structures outlined in Section 2.2. For example, we examine the previously identified inner loop (Figure 3.1) in the cksum trace

$$(ABCDEFGF)^{65536}$$

All adjacent pairs of matrices in the loop body will have the same frequency of 65536. When that is the maximum frequency, the algorithm will compute $A \odot B = X$, and reduce the loop to

$$(XCDEFG)^{65536}$$

After several more passes, the loop body will be reduced to a matrix raised to the power of 65536, which can be rapidly collapsed to a single matrix using a run-length encoding pass.

The ‘encoding’ stage is then formed of a repeated application of pair encoding and run-length encoding (RLE). Pair encoding will reduce each repeat of a pattern to a single matrix, and RLE will collapse repeats into a single matrix. This process repeats until the pair encoder reaches the cutoff threshold.

3.4.3 Multiple pair encoding

While the previously described pair encoding scheme ensures the most frequent adjacent pair is selected at each iteration, computing the frequencies can be expensive. After we evaluate the larger data patterns in the earlier stages of the reduction, the top pair frequency falls dramatically, and simply evaluating the most frequent adjacent pair does not offset the cost of performing the frequency analysis.

We can mitigate this cost by choosing to evaluate many adjacent pairs at an iteration, increasing the amount of work done, and so reducing the number of iterations required. The approach we take is to find the top N most frequent pairs, and set them as *candidates* for replacement. Not all chosen pairs will end up being evaluated, as some may disappear during replacement as they are adjacent to other evaluated pairs.

However, this approach is less accurate and can impact the ability to exploit data patterns. For example, we again inspect cksum’s inner loop (Figure 3.1). We can unroll the loop body once and lay it out like so:

$$(ABCDEFGFGABCDEFGFG)^{32768}$$

Choosing to replace 8 pairs, the algorithm maps $AB \rightarrow H, CD \rightarrow I, \dots$, and the expression is reduced to

$$(HIJKLMN)^{32768}$$

We can repeat this process, unrolling the new loop body again, performing 8 multiplications, and halving the size each time. This will require $8 \cdot \log_2(65536) + 6 = 30$ multiplications *and* $\log_2(65536) = 16$ frequency analyses for a full reduction. By contrast, replacing one pair per iteration requires $6 + \log_2(65536) = 22$ multiplications, and 6 frequency analyses. This is because the reduction happens within the boundaries of the loop body, and reduces it to one matrix, allowing our fast exponentiation algorithm to work.

3.4.4 Block preprocessing

Initially, the expression held one matrix per instruction in the trace, even though the original program trace was provided as a series of basic blocks. We can instead use the information about the program’s basic blocks to build the expression with individual matrices representing the dependency information for an entire basic block, before passing the expression to the encoding stage.

This preprocessing stage multiplies the matrices of an entire basic block, appends the result to the expression, and caches it for future iterations.

This directly avoids reduction occurring across basic block boundaries, and so brings the efficiency of the multiple pair encoder closer to the single pair encoder. An additional benefit

is the reduction in size of the expression passed to the encoder, which not only improves frequency analysis performance, but also reduces memory usage.

However, for loop structures with a period > 1 (for example, `factor` presented in Section 2.2), we still get reduction happening across the boundaries of the loop body.

3.4.5 Variable pair encoding

This variant of the pair encoding scheme attempts to resolve two issues with the multiple pair encoder, while avoiding the poor performance of the single pair encoder in later iterations of the encoding phase.

Poor accuracy with loop structures: As established previously, the multiple pair encoder will not respect loop boundaries, and evaluating too many adjacent pairs will combine several iterations of a loop together, instead of strictly reducing within the loop body.

Unclear parameter effect: The ‘top N pairs’ parameter of the multiple pair encoder tends to be opaque, and its optimal value varies significantly between datasets, as it depends on the internal patterns in the data, not the length of the trace.

The variable pair encoder still replaces multiple pairs per iteration, but chooses the count dynamically. In the frequency analysis stage, the encoder builds a max heap of all unique pairs in the expression, sorted by frequency descending. The encoder iteratively pops pairs from the max-heap, accumulating their frequencies until the sum surpasses a fixed percentage of the total expression size.

We can see how this addresses particular pain points.

- In earlier iterations, loops dominate the expression, and so the frequencies of particular adjacent pairs are very high. The encoder will choose a small number of candidate pairs as their total frequency will exceed the threshold quickly. This avoids replacing enough pairs to reduce across loop boundaries.
- In later iterations, the frequencies of adjacent pairs plummet, and so the encoder will choose many more candidate pairs to exceed the threshold. This ensures enough work is done per iteration to offset the cost of the frequency analysis.
- Finally, the optimal fixed percentage parameter varies much less across different datasets, as evidenced in Section 4.3.2.

3.4.6 Finding flat loops without frequency analysis

While the pair encoding scheme successfully exploits patterns in the data to reduce the total number of multiplications required to reduce the expression, the frequency analysis performed at each iteration is expensive. Both the multiple and variable pair encoding schemes require a $O(n \log n)$ frequency analysis stage, which becomes a bottleneck when processing large program traces.

Subhlok and Hammen [19] identify loop structures in a trace using an approach derived from Crochemore’s algorithm [6]. While their methods are exhaustive and find all PM-repeats,

including overlapping and nested sequences, their worst-case complexity remains $O(n \log n)$, offering no asymptotic benefit over the encoder approach. Our purposes do not require an exhaustive set of overlapping PM-repeats. Instead, we look for a greedy, non-overlapping set of PM-repeats that allows us to easily simplify the trace.

Algorithm 4 is a single-pass, greedy $O(n)$ algorithm that identifies and collapses loops using heuristics in the program trace without relying on frequency analysis. By inserting this algorithm into the reduction pipeline after block preprocessing and before pair encoding, we significantly reduce the size of the expression passed to the pair encoding stage, mitigating the $O(n \log n)$ bottleneck.

```

1 function simplifyTrace(int[] trace) {
2     int[] simplifiedTrace;
3     Map<int, Stack<int>(maxSize=MAX_HISTORY_SIZE)> history;
4     Map<int[], int> cache;
5
6     int i = 0;
7     while (i < trace.size()) {
8         bool detectedLoop = false;
9
10        /* Iterate through recent occurrences backwards */
11        for (prevIndex : history[trace[i]]) {
12            int period = i - prevIndex;
13            if (period > WINDOW_SIZE) break;
14
15            if (verifyMatch(trace, prevIndex, i, period)) {
16                int iterations = countIterations(trace, i, period);
17                int[] loopBody = trace[prevIndex : i];
18
19                /* Reduce and cache */
20                cache[loopBody] = reduce(loopBody);
21                simplifiedTrace.append(exponent(cache[loopBody], iterations));
22
23                /* Jump pointer and reset history to avoid overlap */
24                i = (iterations - 1) * period;
25                history.clear();
26                detectedLoop = true;
27                break;
28            }
29        }
30
31        if (!detectedLoop) {
32            /* Record that we saw this instruction in the trace */
33            history[trace[i]].push(i);
34            simplifiedTrace.append(trace[i]);
35            i++;
36        }
37    }
38
39    return simplifiedTrace;
40 }

```

Algorithm 4: Fast loop-finding algorithm

To maintain a linear time performance, Algorithm 4 uses a bounded history buffer that acts as a sliding window. When it encounters a previously recorded symbol, in this case a matrix pointer, the algorithm searches backwards through recent history; this prioritises the detection of the smallest loop bodies, satisfying primitivity. If a repeated period is confirmed

via `verifyMatch`, the algorithm greedily scans forward to find the number of consecutive iterations. The loop body is reduced to a single matrix, cached for future identical loops, and exponentiated to replace all iterations of the loop. The pointer `i` then advances past the collapsed loop and the history buffer is cleared, which prevent the processing of overlapping PM-repeats and hierarchical nesting.

The helper functions `verifyMatch` and `countIterations` are defined in the appendices (Algorithm 6).

3.5 Parallelisation

In Section 3.4, the analysis tool executed on a single thread. Each component of the pipeline ran sequentially, and when a component decided to multiply a pair of matrices, the operation executed synchronously. We have noted the potential for optimisation of the original problem via parallelisation in Section 1.2, and so this section describes approaches to parallelising the reduction pipeline.

Concurrent data structures are designed for controlling access and modification by multiple threads to shared memory. This project utilises `concurrent_bounded_queue` from Intel’s oneAPI Threading Building Blocks (TBB) library as a producer-consumer queue, which safely blocks consumers when empty. We also make use of a custom lock-free linked list that uses compare and swap (CAS) operations to insert a new node. Insertion is restricted to the head, and we can insert a tombstone sentinel value that causes insertion operations to fail. Pseudocode is given in the appendix (Algorithm 9).

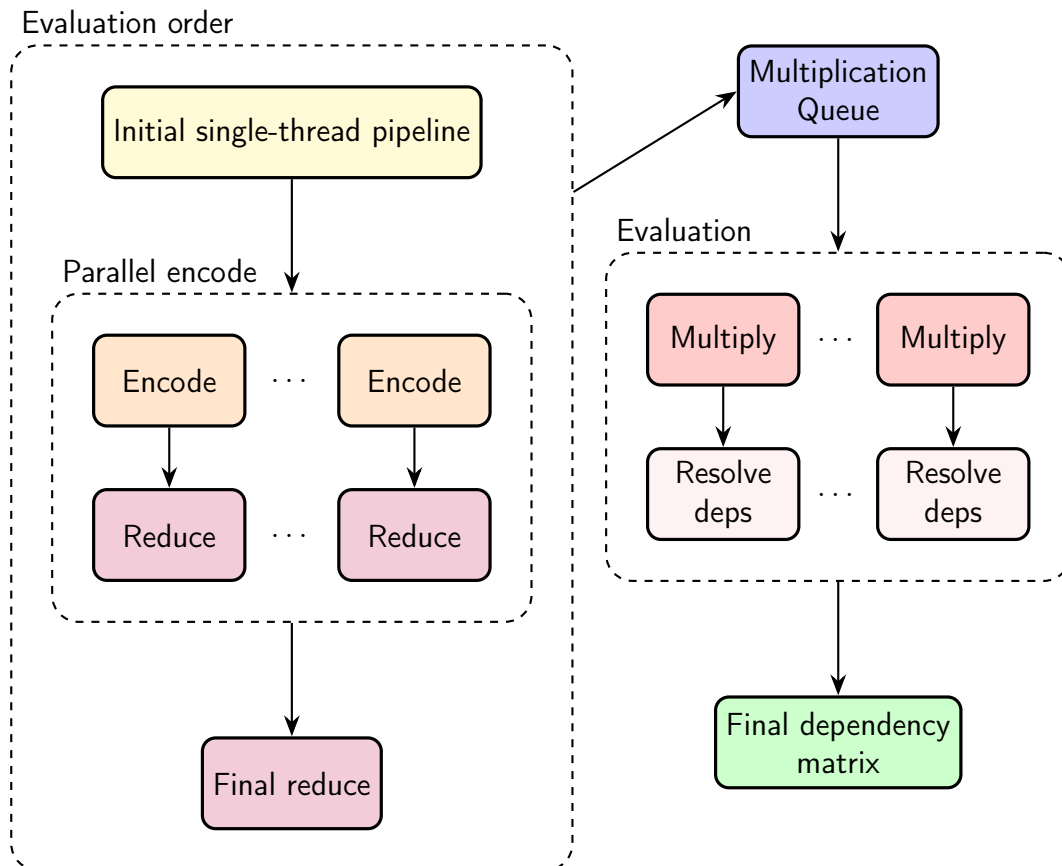


Figure 3.4: Final parallel pipeline.

Figure 3.4 lays out the structure of the parallel program. Splitting the evaluation order and multiplications is covered in Section 3.5.1, and task-parallelism for encoding and multiplication is described in Section 3.5.2.

3.5.1 Pipelined parallelism

Matrix multiplications are done synchronously in the reduction pipeline. However, we observe that the decision to multiply a pair of matrices and the execution of the multiplication are two separate tasks, with the latter dependent on the former.

The sequence of these decisions determines an *evaluation order* for the expression, forming a binary tree with the original matrices at the leaves. Because components select multiplications based only on matrix identifiers (pointers), and not by inspecting the matrix data, an evaluation order can be established without having to wait for multiplications to complete. When a multiplication is scheduled, we allocate an empty matrix with a pointer to it as its identifier. The result is computed asynchronously, and copied back to the empty matrix.

This decouples our program into two primary responsibilities: determining a suitable evaluation order, and subsequently reducing the expression according to the chosen order. This enables an overlapping pipeline where a primary component finds the evaluation order and queues up multiplications, which are then consumed and evaluated by a secondary component.

The first component is largely identical to the original pipeline, except that direct calls to multiply are replaced with calls to queue a new multiplication. The second component, however, is more complex, and has to be careful to execute queued multiplications while respecting evaluation dependencies. More specifically, to perform a multiplication $A \odot B = C$, we must ensure that both A and B are fully evaluated.

We can encapsulate a multiplication operation and all relevant dependencies with the data structure defined in Figure 3.5. All the jobs generated during the analysis will implicitly define the evaluation order. We add a `generatingJob` to the `Matrix` datatype that points to the job that generates its result.

```
struct MatrixJob {
    Matrix* A;
    Matrix* B;
    Matrix* C;
    atomic int unresolvedDeps;
    LockFreeList<Matrix*> dependents;
};
```

Figure 3.5: Multiplication and dependency tracking

When queueing a multiplication, a corresponding job is instantiated and linked to the result matrix. The state of the multiplication operands is then evaluated. If an operand's generating job is still accessible, the system attempts to register the new job by insertion into the lock-free list of dependents on the generating job. If that insertion fails (when a tombstone is encountered at the head of the list), the system safely deduces that the dependency is resolved. If both operands have been resolved, the new job is scheduled for execution.

Scheduled jobs are evaluated by separate threads. After completion of the multiplication, pending dependents must be notified of a dependency resolution. First, a tombstone is atomically inserted at the head of the dependents list to safely indicate to other threads that the job has been completed. Because dependent jobs rely on a compare-and-swap (CAS) operation to insert at the head, the atomic insertion of the tombstone forms a linearization point. The tombstone closes the list, ensuring that no new jobs are added as dependents. The final list is then processed, and the completed job unlinked from the result matrix and discarded.

Pseudocode for queueing and evaluation is given in the appendices (Algorithms 7 and 8).

3.5.2 Work-sharing

Work sharing distributes large computational tasks across several processor cores maximising efficiency. By chunking our workload into many identical subtasks, and using many threads to process the subtasks, we can increase throughput when compared to a single thread.

A thread pool is a collection of pre-initialised ‘worker’ threads that stand ready to perform tasks. Instead of creating a new thread every time a task is to be completed, a program will submit the task to a queue. In our case, we use the `concurrent_bounded_queue`. An idle thread from the pool then picks up the task, executes it, and instead of dying, returns to the pool to wait for the next task. This avoids the overhead associated with creating and destroying threads, improves response times to incoming tasks, and allows us to easily limit the number of threads spawned.

The next two sections present two applications of work-sharing to the analysis tool, both making use of a generic thread-pool implementation. A thread pool is assigned a ‘kernel’, which is a function that each thread applies to a task taken from the queue.

Multiplication scaling

The evaluation of matrix multiplications as described in Section 3.5.1 can be easily parallelised using a thread pool. Once a multiplication job is ready to be executed it can be scheduled by pushing it to the thread pool’s task queue. Multiple threads are able to asynchronously process the evaluation order generated by the first pipeline component.

Increasing the thread count can lead to contention, as threads may try and insert jobs into the same lock-free list simultaneously. This causes the CAS operation to fail for all but one thread, forcing other threads to spin and try again repeatedly. Additionally, heavy access to atomic variables forces CPU caches to invalidate and synchronise across cores to ensure coherence. This traffic impedes linear scaling at high thread counts. The impact of this is evaluated in Section 4.4.1.

Encoding scaling

Additionally, the work done by the encoding and final reduction stages (Sections 3.4.2 and 3.3) can also be distributed amongst several threads. The expression is split into several chunks, and each chunk is reduced to a single matrix through a local application of the encoding and reduction stages by a thread. Once all chunks have been reduced by the thread pool, the final

expression built from the resulting matrices is evaluated. By parallelising and speeding up the evaluation order discovery, we can keep the matrix multiplication thread pool saturated.

However, performing local frequency analysis on smaller chunks of the expression reduces the overall efficiency of the encoding stage. The BPE-adjacent approach performs best with global frequency values as the frequency distribution within a single chunk may not reflect the frequency distribution of the whole expression. This often leads to a worse evaluation order within each chunk, requiring more multiplications to reduce the expression as a whole, but we offset this extra work by utilising more compute when multiplying.

3.6 Accelerator-level parallelism

While a CPU features a handful of powerful, generalised cores, optimised for complex sequential tasks, *Graphical Processing Units* (GPUs) achieve high-throughput compute by employing thousands of simpler, specialised cores using a Single-Instruction, Multiple-Threads (SIMT) [7] architecture. To accelerate highly data-parallel operations like matrix multiplication across many hardware platforms, this project uses OpenCL.

In the OpenCL execution model, the CPU acts as the *Host* which dispatches a kernel program to the GPU (the *Device*). OpenCL organises units of execution into a hierarchy.

NDRange: (N-Dimensional range), the total number of threads executing the kernel.

Work-items: The individual threads executing the kernel, which are organised into **work-groups**.

Work-groups: A collection of work-items that can synchronise and share local memory.

The GPU also features a multi-tier memory hierarchy, with two main levels.

Global memory: The GPU's main memory. It is large, but suffers from high latency and poor bandwidth if not accessed carefully.

Local memory: A small, fast cache shared exclusively among work-items in the same work-group.

Because discrete GPUs are separated from the host by a bus (typically PCIe), data transfers suffer from high latency. To amortise this overhead and fully utilise the bus's bandwidth, tasks are collected into large batches before being dispatched to the device.

3.6.1 Tiling on the GPU

While tiled matrix multiplication was rejected for the CPU in Section 3.2.3 because the 64x64 matrices fit entirely within L1 cache, it is beneficial for the GPU. A naive GPU kernel would force each work-item to read full rows and columns from slow global memory, crippling performance.

Instead, by using a 16x16 tile size, each work-group loads a subset of the matrix from global memory into fast local memory, synchronises, computes partial dot products, and moves on to

the next tile. Although an entire 64x64 matrix would likely be able to fit into a work-group's local memory, tiling allows us to maximise GPU *occupancy*.

GPUs hide resource latency by keeping thousands of threads "in-flight". A *Compute Unit* (CU) in OpenCL is the device component that schedules and executes work-groups (equivalent to a CUDA Streaming Multiprocessor). If a work-group stalls while waiting for global memory, the CU instantly switches to executing another. However, the number of work-groups a CU can manage is limited by the local memory each work-group requires. Tiling minimises the amount of local memory a work-group uses at once, and so several tiles from several matrices can be held in local memory, allowing more independent work-groups to be 'in-flight', than if no tiling was used.

3.6.2 GPU Kernel

The GPU kernel program performs multiplication of dense matrices by following the steps below:

1. Initialise the local memory tile for the work-group.
2. Get the appropriate indices for the work-group and work-item from the 3-dimensional NDRange (representing the index of the matrix and the index of the tile and cell in the matrix).
3. Load the cells of the tile from global memory into local memory. Note that if a work-item is outside the matrix size, it pads the matrix with $-\infty$.
4. Since all work-items in a work group execute synchronously, we place a memory barrier to ensure that all work-items have loaded their cell before performing computation.
5. Compute the dot product of the relevant column and row in the tile, using the max-plus algebra logic.
6. Finally place another memory barrier to ensure all work-items have completed their computation before writing back the result to global memory.

The OpenCL kernel code is given in the appendices (Algorithm 10).

3.6.3 Host to device dispatch

To execute the kernel on the GPU, we need to implement a dispatcher that batches and executes multiplications, and then collects the results, resolving dependencies as needed. Dependency resolution is similar to the logic in Section 3.5.1, but differs slightly as multiplications are batched. The dispatcher replaces the multi-threaded evaluation component in the parallel pipeline from Figure 3.4.

For simplicity, we opted to use a synchronous dispatcher thread and separate a dependency aggregator thread. The synchronous dispatcher thread operates as follows:

1. Collect ready jobs from a queue, up to a maximum batch size.
2. Pack matrix data from matrix objects into flat arrays to be sent to the GPU.

```

.
|- include/
|  |- compute/                | Async CPU/GPU compute, matrix pool.
|  |- expr_simplification/
|  |  |- loop_finding/        | Fast loop finder.
|  |  |- pair_encoding/       | The three pair encoding schemes.
|  |  |- run_length_encoding/ | RLE using fast exponentiation.
|  |  |- tree_reduction/      | Simple tree reduction approach.
|  |- loaders/                | Loading deps, mats and expression.
|  |- matrices/               | Various matrix implementations.
|  |- strategies/             | Baseline, seq and parallel pipelines.
|  |- util/                   | Utility functions, constants, metrics
|- src/
|  |- loaders/                | Loader implementations
|  |- strategies/             | Strategy implementations
|  |- opencl/                 | OpenCL kernels
|  |- CMakeLists.txt
|  |- *_main.cpp              | Various main programs
|-- CMakeLists.txt

```

Figure 3.6: Overview of the project repository and directory structure.

3. Wait for data to be sent to the GPU, computations completed, and results to return.
4. Unpack the matrix data from the flat arrays back into the original matrix objects.
5. Push completed jobs to a separate queue and return to step 1.

The dependency aggregator thread then pops completed jobs off the separate queue and resolves dependents, using the dependency management logic from Section 3.5.1.

While this synchronous model is simple and safe, as multiplications are managed synchronously, it introduces an "idle hardware" problem. While the CPU is processing a batch of matrices, the GPU starves for work. Conversely, when the GPU is computing, the host CPU is blocked, and does nothing.

This presents two clear targets for future optimisation:

1. Use an asynchronous double-buffered queue, where the CPU prepares Batch B while the GPU is actively computing Batch A .
2. Allow for additional free CPU threads to execute multiplications alongside the GPU.

3.7 Repository overview

Figure 3.6. gives a high-level repository overview. Small scripts written for testing, benchmark orchestration and graph plotting are omitted as they do not significantly contribute to the project.

Evaluation

4.1 Methodology

The analysis tools were benchmarked on the program execution traces described in Section 2.2. Benchmarks were executed on two devices, which we call Small and Big, with hardware details in Figure 4.1.

	Small	Big
CPU	AMD Ryzen 9 6900HX	2x Ampere Altra (2x80 cores @ 3GHz)
RAM	16GB DDR5 @ 4800MT/s	32GB DDR4 @ 3200MT/s
GPU	Nvidia GeForce RTX 3080Ti Mobile	N/A

Table 4.1: Hardware specification for each benchmark device. Boost clock was disabled for Small, so each core ran at 3.3Ghz.

Timings and metrics were collected with a custom metrics registry that recorded per-thread events with nanosecond-level timestamps in local thread memory, before flushing to a CSV format at the end of execution. Each configuration of parameters for a benchmark was executed 10 times, and then the median time reported.

4.2 Holistic pipeline

This section compares all notable implementations to the naive baseline (Section 3.1). The notable implementations are the single-threaded pipeline (Figure 3.3), the parallel pipeline (Figure 3.4), and the GPU-enabled pipeline, which is similar to the parallel pipeline except the GPU performs the evaluation instead of a set of CPU cores.

4 encoder threads and 10 compute threads were used for the parallel approach. Fast loop finding was enabled, and the variable pair encoder with $r = 0.4$ was used. For the single-threaded and parallel pipelines, sparse matrix representations were used, but the GPU-enabled pipeline made use of dense matrices. The benchmarks were executed on Small to provide fair comparison with the GPU approach.

Figure 4.1 shows that the single-threaded pipeline offers better performance than the naive baseline in almost all cases. The data-driven optimisations successfully reduce work enough to make matrix multiplication a viable choice, except for sort, where the complex program structure is hard to exploit.

Across every single program trace, the fully parallel pipeline is 2 to 7 times faster than the naive baseline. Parallelisation clearly accelerates reduction, but trace loading is unchanged; the sequential I/O involved visualises Amdahl’s Law. Loading the trace dominates the program’s lifecycle, implying diminishing returns from further parallelisation. Furthermore,

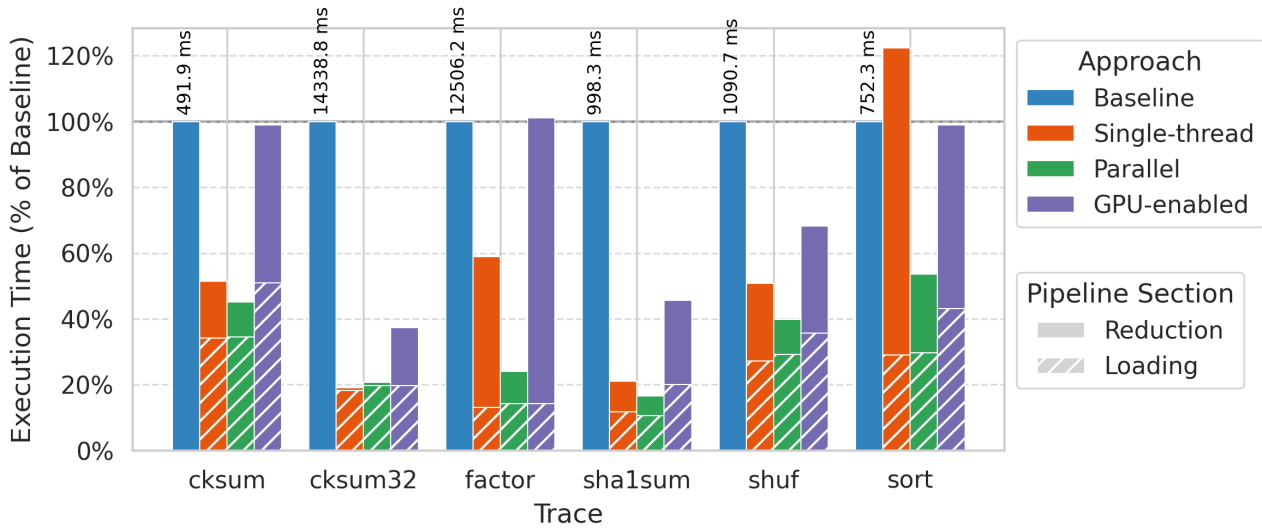


Figure 4.1: Comparing the speedup for different implementations, executed on the Small machine.

Dataset	NB	Single	Para.	GPU
cksum	11.2	43.3	49.8	582
cksum32	11.1	551	560	1052
factor	11.1	1601	906	11616
shalsum	11.4	46.0	55.7	606
shuf	11.4	79.6	79.6	623
sort	11.8	182	203	714

Table 4.2: Memory usage breakdown for each approach (MB, maximum resident set size)

the GPU approach is almost universally slower than both the single-threaded and parallel approaches. This is likely due to the poor GPU throughput and throttling of the memory bandwidth, as dense matrix data is constantly being transferred between the CPU and GPU. This adds increased contention for other threads performing loading and encoding.

Table 4.2 provides an overview of memory footprints. Matrix based approaches predictably use more memory, as not only does the system store more data in matrices but also requires the entire trace to be loaded simultaneously for reduction, whereas the baseline streams the trace from disk and discards it after processing. The GPU approach has the largest footprint. It uses dense matrix representations, and its poor throughput creates a large backlog of pending multiplications. Because result memory is allocated upon queueing, the system is forced to hold memory for both operands and the pending result for extended periods of time while the GPU clears the backlog.

4.3 Algorithmic and data-driven optimisations

This section evaluates the impact of the data-driven optimisations presented in Sections 3.2 and 3.4, to assess the PE requirement from Table 2.2. Benchmarks are executed on the Small machine for increased single-thread performance, and use the single-threaded pipeline from Figure 3.3, in some cases with components excluded to assess their impact.

4.3.1 Matrix multiplications

We first evaluate our matrix implementations presented in Section 3.2. This benchmark made use of the Google Benchmark framework, and executed multiplications on matrices of size 64×64 of varying sparsity. The matrices were randomly generated, by generating a set number of unique coordinates and assigning values to them, and filling the rest of the matrix with $-\infty$.

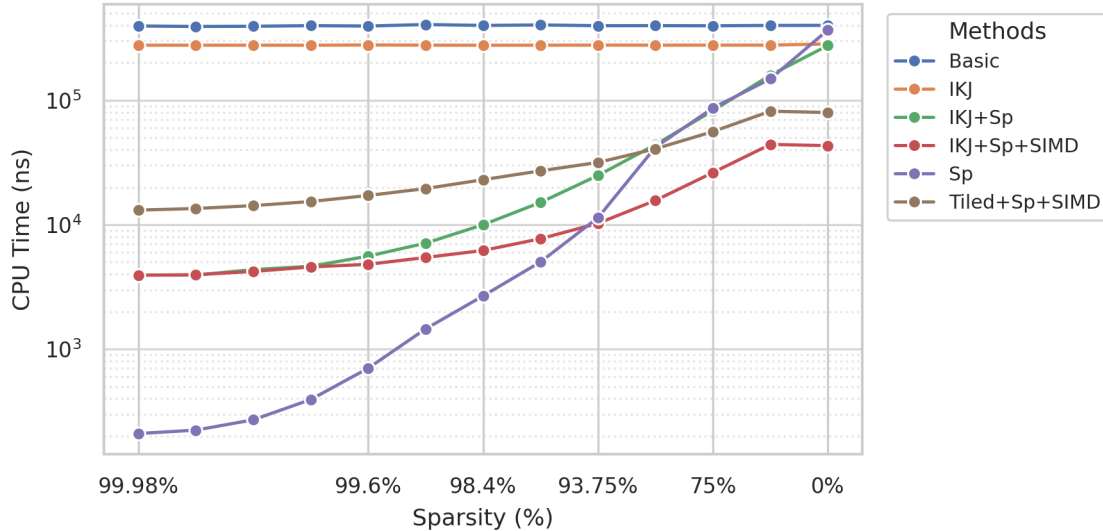


Figure 4.2: Comparing runtime of matrix multiplication approaches, for 64×64 matrices of varying sparsity.

Figure 4.2 shows our basic implementation has the worse performance overall. Swapping the order of our inner loops improves performance, and taking advantage of sparsity through the optimisation mentioned in 3.2.2 massively reduces execution time for sparser matrices. Introducing SIMD extensions speeds up the implementation for denser matrices, where contiguous values are more likely to appear and so are optimised by the vectorised computation. We can see that tiled multiplication introduces overhead and does not offer any benefit over our other dense matrix implementation.

The sparse matrix implementation offers better performance for sparsity $> 93.75\%$ but rapidly deteriorates for denser matrices, and is close to the basic implementation for non-sparse matrices. Considering the distribution of sparsity in our datasets, shown in Section 3.2 and Appendix C, sparse matrix multiplications offer the best performance for CPU-based multiplications, and so are used in all following benchmarks.

4.3.2 Encoder performance and compression

Evaluating the encoding strategies reveals a fundamental trade-off between the computational cost of frequency analysis and the optimality of the resulting evaluation tree.

Figure 4.3 shows off the execution time of the encoder and tree-reduction stages of the pipeline for various parameters. We immediately observe that the execution time curves for fixed pair counts are highly erratic and dataset-dependent. For instance, the optimal p for cksum32 is vastly different from the optimal p for factor. Using a fixed p is brittle because it does not scale with the size or entropy of the execution trace. Replacing 2^5 pairs in a large,

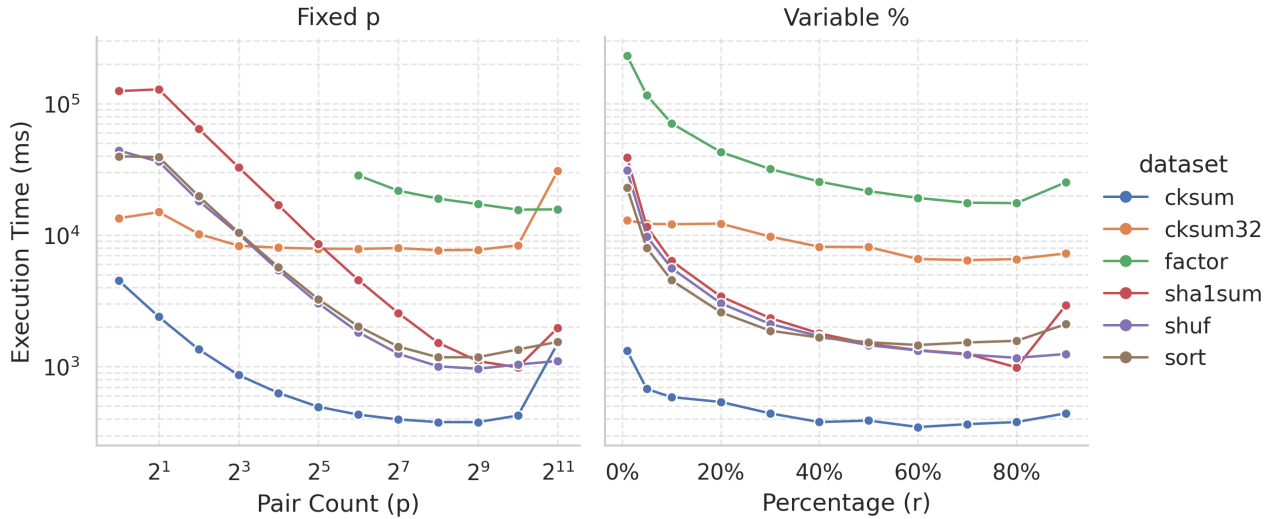


Figure 4.3: Comparing runtime of encoding and reduction stages on different datasets, for different encoder parameters, excluding block preprocessing and loop finding.

high-entropy trace may do close to nothing (wasting time on frequency analyses), whereas replacing 2^5 pairs in a small trace with lots of patterns may aggressively combine matrices, destroying the compression ratio, therefore a "universal sweet spot" for p does not exist.

Compare this to the execution time curves for the variable parameter. They are much smoother and form a gentle U shape across most datasets. The best execution time ranges between $r = 0.6$ and $r = 0.8$. Because this parameter scales with the size and complexity of the expression, it balances the workload. While the best performance is not as good as the fixed pair encoder, the dynamic threshold yields a more stable sweet spot, regardless of the input program, as hypothesised in Section 3.4.5.

The reason for the high execution times for $p = 1$ shown in Figure 4.3 can be visualised further in Figure 4.5. The $p = 1$ reduction profile shows periods of minimal reduction, where the algorithm carefully replaces the most frequent matrix pair. While this is theoretically optimal, each iteration requires an expensive frequency analysis, which dominating the runtime. This contrasts heavily to the reduction profiles of $p = 256$ and $r = 0.4$, which plunge steeply. They achieve slightly worse compression ratios, but do so with one to two orders of magnitude fewer frequency analyses.

Figure 4.4 shows the compression ratio achieved by the encoding stage for each parameter. We can see that increasing the number/proportion of replacements per iteration reduces the efficiency of the encoder, as the algorithm is forced to group sub-optimal, lower-frequency pairs, resulting in a poorer evaluation tree. This is particularly apparent for the variable pair encoder, as the replacement count increases aggressively in later iterations as larger patterns are exploited.

Figure 4.5 shows the impact of block-preprocessing and fast loop finding on the encoding stage. For highly repetitive traces like `cksum`, the optimisations collapse the expression before passing it to the encoder, allowing it to begin its work on an expression that is already a fraction of the size. This effectively skips the early, slow iterations required to compress the loop structures.

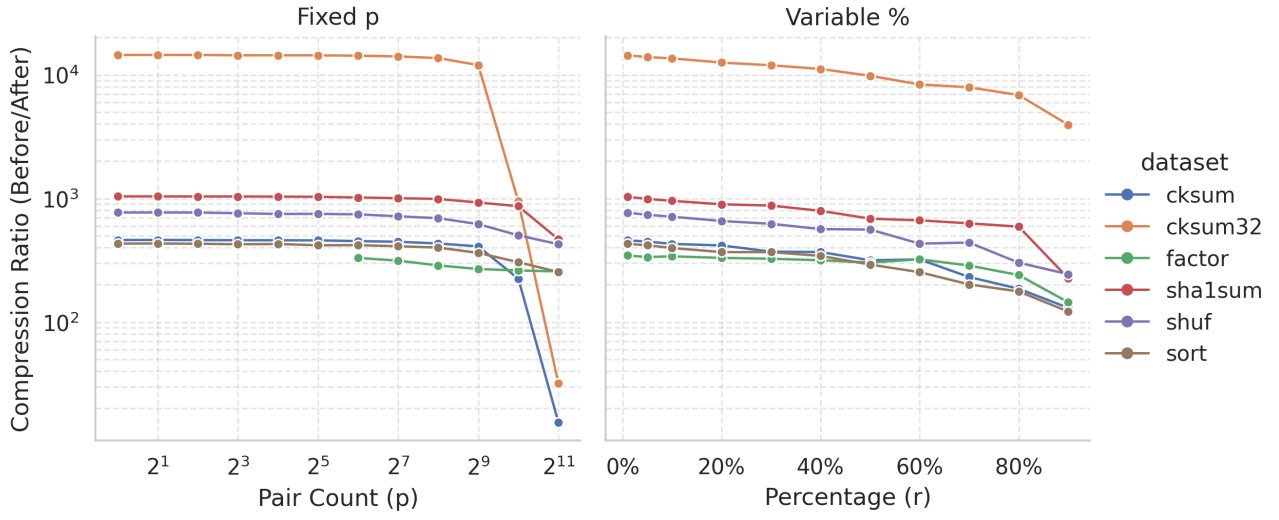


Figure 4.4: Comparing compression ratio of encoder on different datasets, for different encoder parameters, excluding block preprocessing and loop finding.

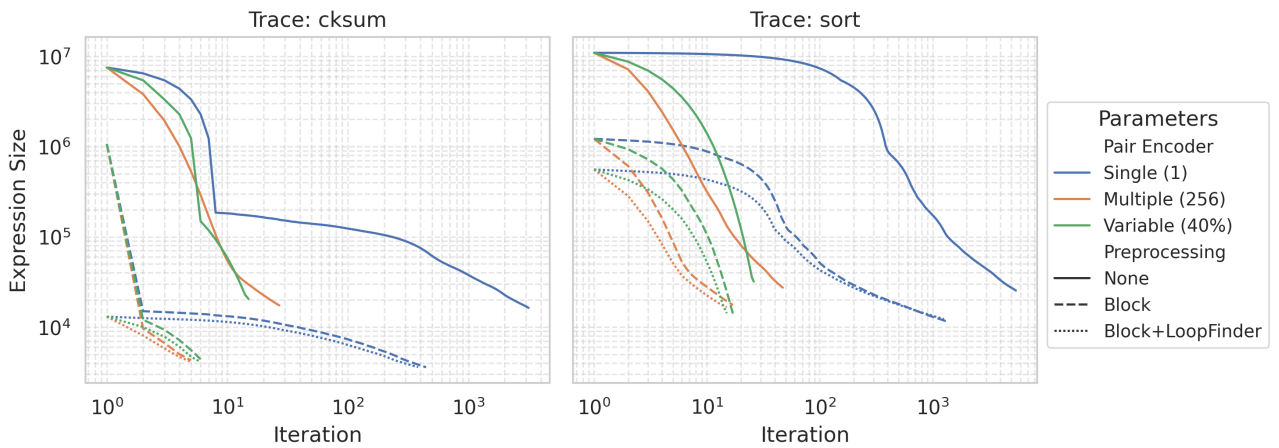


Figure 4.5: Comparing reduction behaviour of encoder on different datasets, for representative parameters.

4.3.3 Single-threaded pipeline

This section evaluates the single-threaded pipeline shown in Figure 3.3, compared to the same pipeline without the block preprocessing and fast loop finding stages.

Dataset	Base Encoder		+ Block Prep			+ Fast Loop Finder			
	PE	FR	BP	PE	FR	BP	LF	PE	FR
cksum	10,876	20,413	15,858	2,082	4,367	15,858	434	1,332	4,105
cksum32	10,740	21,074	15,851	2,134	4,235	15,851	472	1,360	4,045
factor	99,764	697,842	17,429	71,156	670,298	17,429	7,988	73,266	687,417
shalsum	15,350	21,411	18,585	2,326	4,488	18,585	581	1,478	4,695
shuf	15,750	28,958	17,776	5,118	9,186	17,776	3,578	3,714	9,605
sort	20,118	31,922	19,270	5,404	14,003	19,270	9,842	4,190	14,203

Table 4.3: Breakdown of matrix multiplications across pipeline configurations, using variable encoder with $r = 0.4$. **BP**: Block Preprocessing, **LF**: Loop Finder, **PE**: Pair Encoding, **FR**: Final Tree Reduction.

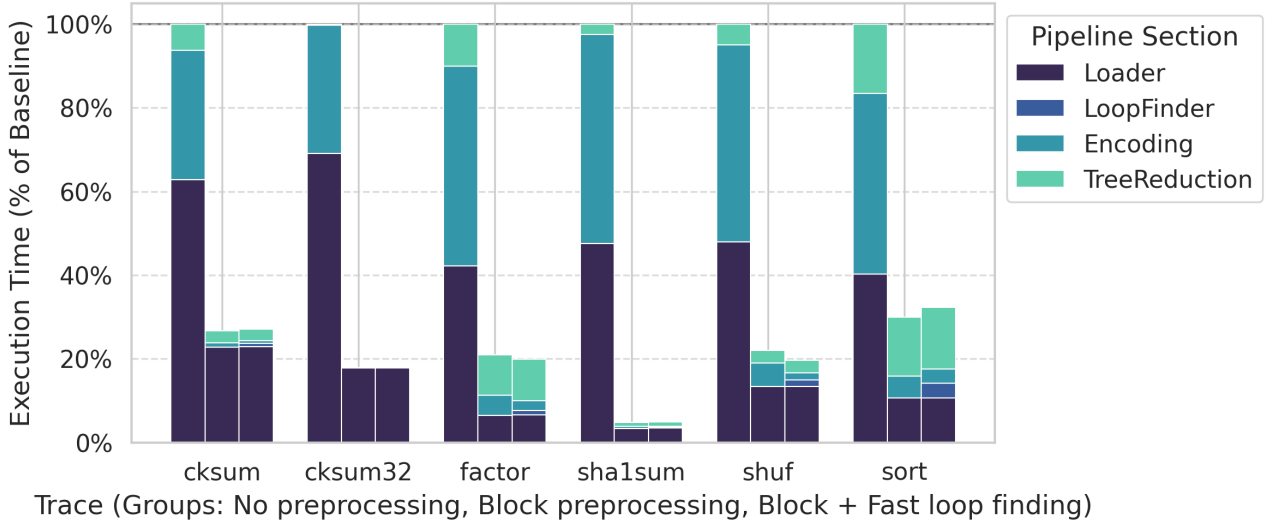


Figure 4.6: Comparing the speedup for block preprocessing and fast loop finding, using variable encoder with $r = 0.4$

Figure 4.6 quantifies the huge impact of block preprocessing on execution time. Across the board, execution time drops by 60% to 80%. This is primarily down to three different reasons:

1. The loader iterates and caches at the block-level when loading the trace, reducing the amount of work it needs to do.
2. The size of the expression passed to the encoder is much smaller, reducing the cost of the $O(n \log n)$ frequency analyses. This can be seen in Table 4.3, where the number of multiplications the encoder and final reduction make falls.
3. Because matrices represent block-level dependencies, the huge 1-block loops of cksum, cksum32, and shalsum can be rapidly collapsed using the first run-length encoding pass in Figure 3.3. This almost entirely eliminates the encoding and reduction phases, and loading the trace becomes the primary bottleneck.

On the other hand, the fast loop finding stage delivers mixed results. Execution time is largely unchanged by including fast loop finding for cksum, cksum32, and shalsum. This is backed up by Table 4.3, where the loop finder does very few multiplications, as most of the expression is already reduced by the first RLE stage. Performance improves for the shuf and factor traces, but interestingly, the total number of multiplications done increases. The greedy nature of the loop finder led to a worse evaluation order, but performing more multiplications is cheaper compared to the cost of the frequency analyses in the encoding stage. This trade-off does not translate to sort, however, as the performance is worse, and many more multiplications are made. This implies that the greedy approach has much worse efficiency for this trace and the complex branch structure of the underlying merge sort algorithm is better suited to a pure pair encoding approach.

Overall, I believe this single-threaded pipeline satisfies the success criteria for the PE requirement in Table 2.2.

4.4 Concurrency and GPU analysis

This section breaks down the parallel optimisations presented in Sections 3.5 and 3.6, to assess the the MP and GA project requirements from Table 2.2.

Benchmarks are performed using the multi-threaded pipeline from Figure 3.4. We focus on the factor trace, as it required the largest number of multiplications to reduce the expression.

4.4.1 Concurrency

Concurrency benchmarks are executed on the Big machine to show off scaling properties on a wider range of thread configurations.

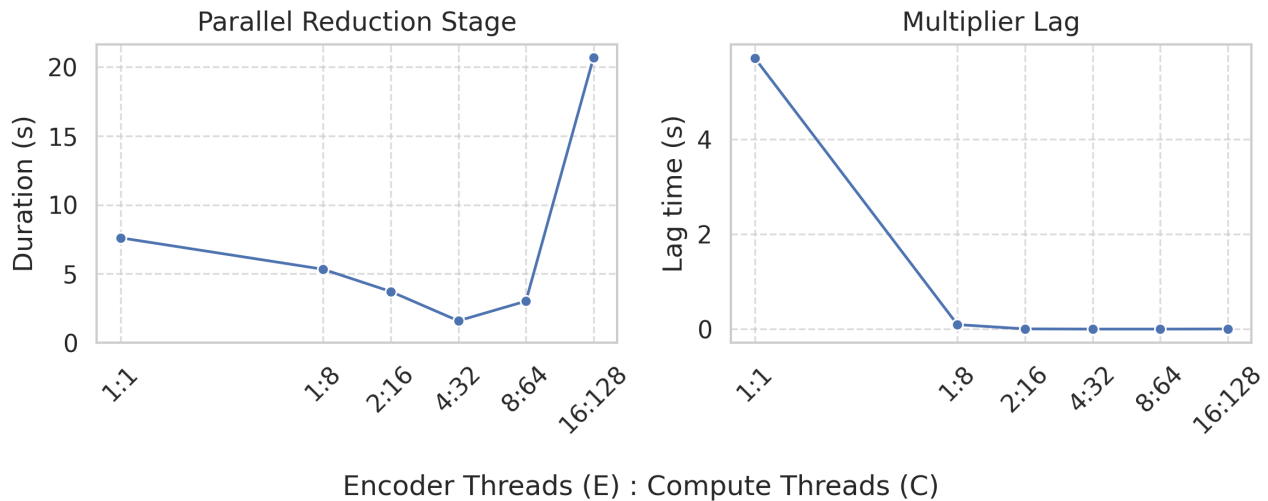


Figure 4.7: Multi-threaded pipeline component runtime and multiplier lag time for various thread configurations.

Figure 4.7 immediately shows the impact of introducing an asynchronous architecture on the runtime of the parallel pipeline components. By increasing the number of threads, runtime drops from 7.5s down to 1.5s. This satisfies the success criteria for the MP requirement, as

the program clearly scales with increased thread counts. The lag time of the pipeline is the time between when the full evaluation order is determined and when the expression is fully reduced. This is eliminated as the number of compute threads relative to the number of encoding threads is increased, showing that each encoder thread generates enough multiplication jobs for several compute threads.

Performance worsens slightly at the 8:64 thread mark. This is not due to multiplication performance, as active throughput steadily increases up to 8:64 and dependency management overhead remains at $< 10\%$, as shown in Figure 4.8. However, we can see that encoder efficiency is impacted. The proportion of time the encoder threads are doing useful work falls steadily as thread counts increase. Encoder threads spend increasingly more time on the dependency resolution involved when queueing multiplication jobs, and so the analysis and reduction of the matrix expression does not get any faster. This is most likely due to increased cache contention, as the number of threads doing memory-heavy matrix multiplications has increased and shared caches are oversaturated.

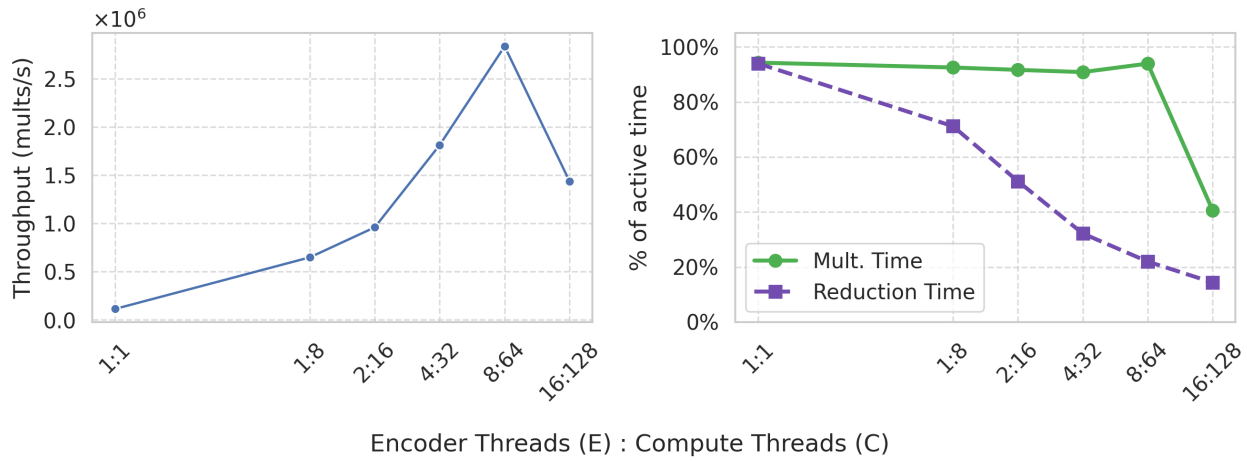


Figure 4.8: Matrix multiplication throughput and overhead for various thread configurations.

Performance at the 16:128 thread configuration collapses due to the consequences of Non Uniform Memory Access (NUMA). While the Big machine supports up to 160 hardware threads, they are physically distributed across a dual-socket architecture. Scaling to more than 80 threads schedules them across both processors. Because the program is NUMA oblivious, threads on processor 0 constantly attempt to read and modify jobs and dependencies residing in the cache of processor 1 and vice-versa. This cross-socket contention triggers huge cache coherency traffic across the processor interconnect, as shared cache lines are repeatedly invalidated and transferred between both processors. The cores are subject to severe memory stalls and throughput slumps.

4.4.2 GPU analysis

GPU benchmarks are executed on the Small machine to make use of the 3080Ti Mobile.

Table 4.4 details a 4096-multiplication batch dispatched to the OpenCL kernel (Section 3.6.2). The total time of 41.7ms yields a throughput of $\sim 100,000$ multiplications per second. Compared to CPU scaling (Figure 4.8), this mirrors the throughput of a single CPU thread, and is eclipsed by the full use of 12-15 threads.

Section	Time taken (ms)	Percentage
Pack	15.9	38.1%
Send	9.92	23.7%
Compute	1.70	4.06%
Receive	4.78	11.4%
Unpack	9.46	22.7%
Total	41.7	100%

Table 4.4: GPU timeline breakdown for a batch of 4096 multiplications.

Data management and transfer overhead consumes $\sim 96\%$ of the execution time. As matrices are allocated dynamically on the heap (see Section 3.3), their non-contiguous addresses force the CPU to chase scattered pointers and individually copy each matrix’s data during packing and unpacking. This is exacerbated by the use of a synchronous pipeline. Implementing a double-buffered approach would allow the CPU to pack and transfer batch B while the GPU computes the previously prepared batch A , effectively hiding the data management overhead behind computation.

The success criteria for the requirement GA in Table 2.2 has been met, as we successfully offloaded multiplications to the GPU, although throughput never overtook overhead.

Conclusions

The matrix-based approach offers a direct performance improvement (a 2-7x speedup, see Section 4.2) over a classical baseline when conducting ILP limit studies. All the minimum success criteria were met, and nearly all original extensions were implemented with varying levels of success. The associativity of the matrix expression shows clear pathways for both data-driven and parallel optimisations.

5.1 Lessons learned

The theory behind the matrix-based approach was well-established and understood going into the project. However, pattern exploitation was not well-defined in the project proposal, and so much of the work in Section 3.4 was exploratory and took more time than expected. This left little time to deal with the steep learning curves of multithreaded and GPU programming, and highlighted the importance of domain knowledge and tool preparation. Additionally, these hardware optimisations were developed and optimised locally on my laptop, and not iteratively tested on suitable hardware. For the multithreaded approach, this meant that utilising more cores on the server hardware was not impactful enough to overtake the fewer, yet more powerful laptop cores.

Evaluation proved to be a tricky part of the project. While its importance was understood, my methodology was initially underdeveloped and so I was unable to thoroughly evaluate my progress as I made changes. Instead, much of the project was benchmarked all at once during the writeup, and this made fixing critical performance bugs more difficult.

5.2 Future work

Listed are shortcomings of the analysis tool that could be addressed in the future.

- The GPU pipeline was inefficient. The memory layout could be optimised alongside a new double-buffered dispatch architecture to reduce data management overhead. The CPU could also perform multiplications concurrently, maximising throughput.
- I/O and parsing form a sequential bottleneck for the analysis tool when loading the initial expression. An alternative binary-based format could be developed to speed up loading and reduce the bottleneck.
- The tool only tracked register dependencies. While this is still useful for smaller subtraces that keep their working set in registers, programs will often spill registers to memory. This occurred with all the sample traces, and so reported ILP limits were higher than actual values. The impact of the data-driven optimisations could be assessed on more complex program traces involving memory dependencies.

Overall, the project achieved its goals, and was an interesting problem covering many domains.

Bibliography

- [1] Todd M. Austin and Gurindar S. Sohi. Dynamic dependency analysis of ordinary programs. In *Proceedings of the 19th Annual International Symposium on Computer Architecture*, ISCA '92, page 342–351, New York, NY, USA, 1992. Association for Computing Machinery. ISBN 0897915097. doi: 10.1145/139669.140395. URL <https://doi.org/10.1145/139669.140395>.
- [2] Richard Bellman. *Dynamic Programming*. Dover Publications, 1957. ISBN 9780486428093.
- [3] Alexandra Chadwick. On tropical semirings and the limits of instruction-level parallelism. <https://www.cl.cam.ac.uk/~awc32/tropical-ilp.pdf>, 2025.
- [4] Alexandra W. Chadwick, Márton Erdős, Utpal Bora, Akshay Bhosale, Bob Lytton, Yuxin Guo, Richard Cooper, Giacomo Gabrielli, and Timothy M. Jones. The future of instruction-level parallelism (ILP). In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 350–352, 2025. doi: 10.1109/ISPASS64960.2025.00040.
- [5] Dai Clegg and Richard Barker. *Case Method Fast-Track: A Rad Approach*. Addison-Wesley Longman Publishing Co., Inc., USA, 1994. ISBN 020162432X.
- [6] Max Crochemore. An optimal algorithm for computing the repetitions in a word. *Information Processing Letters*, 12(5):244–250, 1981. ISSN 0020-0190. doi: [https://doi.org/10.1016/0020-0190\(81\)90024-7](https://doi.org/10.1016/0020-0190(81)90024-7). URL <https://www.sciencedirect.com/science/article/pii/0020019081900247>.
- [7] Michael J Flynn. Very high-speed computing systems. *Proceedings of the IEEE*, 54(12): 1901–1909, 1966.
- [8] Free Software Foundation. *GNU Coreutils - Core GNU utilities*, 2025. URL <https://www.gnu.org/software/coreutils/>. Version 8.32.
- [9] Philip Gage. A new algorithm for data compression. *C Users J.*, 12(2):23–38, February 1994. ISSN 0898-9788.
- [10] Bernard Goossens, Philippe Langlois, David Parello, and Eric Petit. PerPI: A Tool to Measure Instruction Level Parallelism. In *Applied Parallel and Scientific Computing*, volume LNCS of *10th International Conference, PARA 2010, Reykjavík, Iceland, June 6-9, 2010, Revised Selected Papers, Part I*, pages 270–281. Springer Berlin Heidelberg, 2012. doi: 10.1007/978-3-642-28151-8_27. URL <https://hal-lirmm.ccsd.cnrs.fr/lirmm-01349703>.
- [11] Fred G. Gustavson. Two fast algorithms for sparse matrices: Multiplication and permuted transposition. *ACM Trans. Math. Softw.*, 4(3):250–269, September 1978. ISSN 0098-3500. doi: 10.1145/355791.355796. URL <https://doi.org/10.1145/355791.355796>.
- [12] John L. Hennessy and David A. Patterson. *Computer Architecture, Fifth Edition: A Quantitative Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 5th edition, 2011. ISBN 012383872X.

- [13] Nikolai K. Krivulin. The max-plus algebra approach in modelling of queueing networks, 2012. URL <https://arxiv.org/abs/1212.0895>.
- [14] Matthew A. Postiff, David A. Greene, Gary S. Tyson, and Trevor N. Mudge. The limits of instruction level parallelism in SPEC95 applications. *SIGARCH Comput. Archit. News*, 27(1):31–34, March 1999. ISSN 0163-5964. doi: 10.1145/309758.309771. URL <https://doi.org/10.1145/309758.309771>.
- [15] J.P. Shen and M.H. Lipasti. *Modern Processor Design: Fundamentals of Superscalar Processors*. Waveland Press, 2013. ISBN 9781478610762. URL <https://books.google.co.uk/books?id=ffQqAAAAQBAJ>.
- [16] V. STRASSEN. Gaussian elimination is not optimal. *Numerische Mathematik*, 13: 354–356, 1969. URL <http://eudml.org/doc/131927>.
- [17] David W. Wall. Limits of instruction-level parallelism. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS IV, page 176–188, New York, NY, USA, 1991. Association for Computing Machinery. ISBN 0897913809. doi: 10.1145/106972.106991. URL <https://doi.org/10.1145/106972.106991>.
- [18] Ryan Wandsnider. The fundamental theorem of algebra over the tropical semiring. *University of Chicago Undergraduate REU*, August 2020. URL <https://math.uchicago.edu/~may/REU2020/REUPapers/Wandsnider.pdf>.
- [19] Qing Xu, Jaspal Subhlok, and Nicholas Hammen. Efficient discovery of loop nests in execution traces. In *2010 18th International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 193–202. IEEE, 2010.

Mathematical definitions and proofs

Definition A.0.1 (Semiring). A semiring [18] is a set R along with two operations, addition $(+)$ and multiplication $(\cdot)$, which satisfy the following axioms:

1. Addition is associative and commutative.
2. Multiplication is associative.
3. There exists an additive identity, which we usually call $\mathbb{0}$. Additionally, $\mathbb{0}$ annihilates R , that is, for all $x \in R$, $\mathbb{0} \cdot x = x \cdot \mathbb{0} = \mathbb{0}$.
4. There exists a multiplicative identity, which we usually call $\mathbb{1}$.
5. Multiplication left and right distributes over addition.

Theorem 2.1.1 (Matrix exponentiation results in longest paths). For any integer $q > 0$, the matrix X^q has the entry $x_{ij}^{(q)} = p$ only if there is a path of length p with at most q edges from i to j , and that path is the longest such path.

Proof. We proceed by induction on q , the maximum number of edges in the path.

Base case ($q = 1$): Consider the matrix $X^1 = X$. By the definition of an adjacency matrix, the entry $x_{ij}^{(1)}$ is exactly the weight of the direct edge from node i to node j . If no such edge exists, then $x_{ij}^{(1)} = -\infty$. Thus, the base case holds: the entries represent their longest paths consisting of at most one edge.

Inductive step: Assume the claim holds for q . That is the matrix X^q has entries x_{ij}^q representing the longest path from i to j using at most q edges. We show that the claim holds for $q + 1$. By definition, $X^{q+1} = X^q \odot X$. The entry $x_{ij}^{(q+1)}$ is calculated as:

$$x_{ij}^{(q+1)} = \bigoplus_{k=1}^n (x_{ik}^{(q)} \odot x_{kj})$$

Translating this to the original operators:

$$x_{ij}^{(q+1)} = \max_{1 \leq k \leq n} (x_{ik}^{(q)} + x_{kj})$$

This lines up perfectly with the dynamic programming approach to finding the longest path. To find a longest path from node i to node j , it checks every intermediate node k . It takes the longest known path from i to k of at most q edges (by the inductive hypothesis), and adds the direct edge cost from k to j . By taking the maximum over all intermediate nodes k , we find the absolute longest path from i to j of at most $q + 1$ edges. Thus the claim holds for all integers $q > 0$. $\square$

Additional code listings

```

1 function multiply(SparseMatrix<m,n> A, SparseMatrix<n,p> B) {
2   int[p] x = [-∞,...,-∞];
3   int[p] xb = [-1,...,-1];
4   Queue<int> iCols;
5   SparseMatrix<m,p> C;
6
7   IC[0] = 0;
8
9   for (i : [0,...,m-1]) {
10    for (jp : [IA[i],...,IA[i+1]-1]) {
11      int j = JA[jp];
12      int a = VA[jp];
13
14      for (kp : [IB[j],...,IB[j+1]-1]) {
15        int k = JB[kp];
16        int b = VB[kp];
17
18        if (xb[k] != i) {
19          xb[k] = i;
20          x[k] = a + b;
21          iCols.push(k);
22        } else {
23          x[k] = max(x[k], a + b);
24        }
25      }
26    }
27
28    while (!iCols.empty) {
29      int col = iCols.pop();
30      // Assume dynamically allocated arrays
31      JC.push(col);
32      VC.push(x[col]);
33    }
34
35    IC[i+1] = JC.size;
36  }
37
38  return C;
39 }

```

Algorithm 5: Sparse-Sparse matrix multiplication

```

1 function verifyMatch(int[] trace, int startA, int startB, int length) {
2     /* Do a quick boundary check and exit if tails don't match */
3     if (startB + length > trace.size()) return false;
4     if (trace[startA + length - 1] != trace[startB + length - 1]) return false
5     ;
6     /* Verify full sequence */
7     for (k : [0,...,length-2]) {
8         if (trace[startA + k] != trace[startB + k]) {
9             return false;
10        }
11    }
12    return true;
13 }

```

Listing B.1: Verifying trace ranges match

```

1 function countIterations(int[] trace, int currentPos, int period) {
2     int iterations = 2; // Account for original body and first match
3     int scanPos = currentPos + period;
4
5     for (scanPos + period <= trace.size()) {
6         if (verifyMatch(trace, currentPos, scanPos, period)) {
7             iterations++;
8             scanPos += period;
9         } else {
10            break;
11        }
12    }
13    return iterations;
14 }

```

Listing B.2: Greedily finding all iterations

Algorithm 6: Helper functions for the fast loop finder. Standard array-scanning logic.

```

1 concurrentQueue<MatrixJob> readyJobs;
2
3 /* Queueing up a new multiplication */
4 function enqueueMultiplication(MatrixJob newJob) {
5     newJob.unresolvedDeps = 2;
6     newJob.C.generatingJob = newJob;
7
8     checkDependency(A,C);
9     checkDependency(B,C);
10
11     if (newJob.unresolvedDeps == 0) {
12         readyJobs.push(newJob);
13     }
14 }

```

Listing B.3: Queueing Logic

```

1 // FAS: Atomic fetch and subtract
2
3 function checkDependency(Matrix* dependency, Matrix* dependent) {
4     MatrixJob job = dependency.generatingJob;
5
6     if (job != nullptr
7         && job.dependents.insert(dependent.generatingJob)) {
8         return;
9     }
10
11     FAS(newJob.unresolvedDeps, 1);
12 }

```

Listing B.4: Dependency Processing

Algorithm 7: Matrix queueing and dependency management logic.

```

1 concurrentQueue<MatrixJob> readyJobs;
2
3 void executeMultiplication(MatrixJob job) {
4     *(job.C) = *(job.A) * *(job.B);
5
6     Node<T> head = job.dependents.close();
7
8     /* traverses the dependents list,
9        decrementing unresolvedDeps and queueing a job if ready. */
10    notifyAndQueueDependents(head);
11
12    job.C.generatingJob = null;
13 }
14

```

Algorithm 8: Asynchronous matrix multiplication and dependency management logic.

```
1 // CAS: Atomic compare and swap
2
3 bool insert(T item) {
4     Node<T> new = Node<T>(item);
5
6     while (true) {
7         Node<T>* currHead = head;
8         if (currHead == tombstone) {
9             return false;
10        }
11
12        new.next = currHead;
13        if (CAS(currHead, new)) {
14            return true;
15        }
16    }
17 }
18
19 Node<T> close() {
20     Node<T> tmp = head;
21     CAS(head, tombstone);
22     return tmp;
23 }
24
```

Algorithm 9: Lock-free list operations.

```

1 #define TILE_SIZE 16
2 __kernel void batched_max_plus_tiled(__global const int* A_batch,
3                                     __global const int* B_batch,
4                                     __global int* C_batch,
5                                     const int N,
6                                     const int negative_infinity) {
7     __local int As[TILE_SIZE][TILE_SIZE];
8     __local int Bs[TILE_SIZE][TILE_SIZE];
9
10    int bx = get_group_id(0);
11    int by = get_group_id(1);
12    int tx = get_local_id(0);
13    int ty = get_local_id(1);
14    int batch_idx = get_group_id(2);
15
16    int row = by * TILE_SIZE + ty;
17    int col = bx * TILE_SIZE + tx;
18    int offset = batch_idx * (N * N);
19
20    int max_val = negative_infinity;
21    int num_tiles = (N + TILE_SIZE - 1) / TILE_SIZE;
22
23    for (int t = 0; t < num_tiles; ++t) {
24        if (row < N && (t * TILE_SIZE + tx) < N) {
25            As[ty][tx] = A_batch[offset + row * N + t * TILE_SIZE + tx];
26        } else {
27            As[ty][tx] = negative_infinity;
28        }
29
30        if ((t * TILE_SIZE + ty) < N && col < N) {
31            Bs[ty][tx] = B_batch[offset + (t * TILE_SIZE + ty) * N + col];
32        } else {
33            Bs[ty][tx] = negative_infinity;
34        }
35
36        barrier(CLK_LOCAL_MEM_FENCE);
37
38        for (int k = 0; k < TILE_SIZE; ++k) {
39            int a_val = As[ty][k];
40            int b_val = Bs[k][tx];
41
42            if (a_val != negative_infinity && b_val != negative_infinity) {
43                int current_path_length = a_val + b_val;
44                if (current_path_length > max_val) {
45                    max_val = current_path_length;
46                }
47            }
48        }
49        barrier(CLK_LOCAL_MEM_FENCE);
50    }
51
52    if (row < N && col < N) {
53        C_batch[offset + row * N + col] = max_val;
54    }
55 }

```

Algorithm 10: Tiled GPU kernel for computing batches of matrix multiplications

Distribution of matrix sparsity in datasets

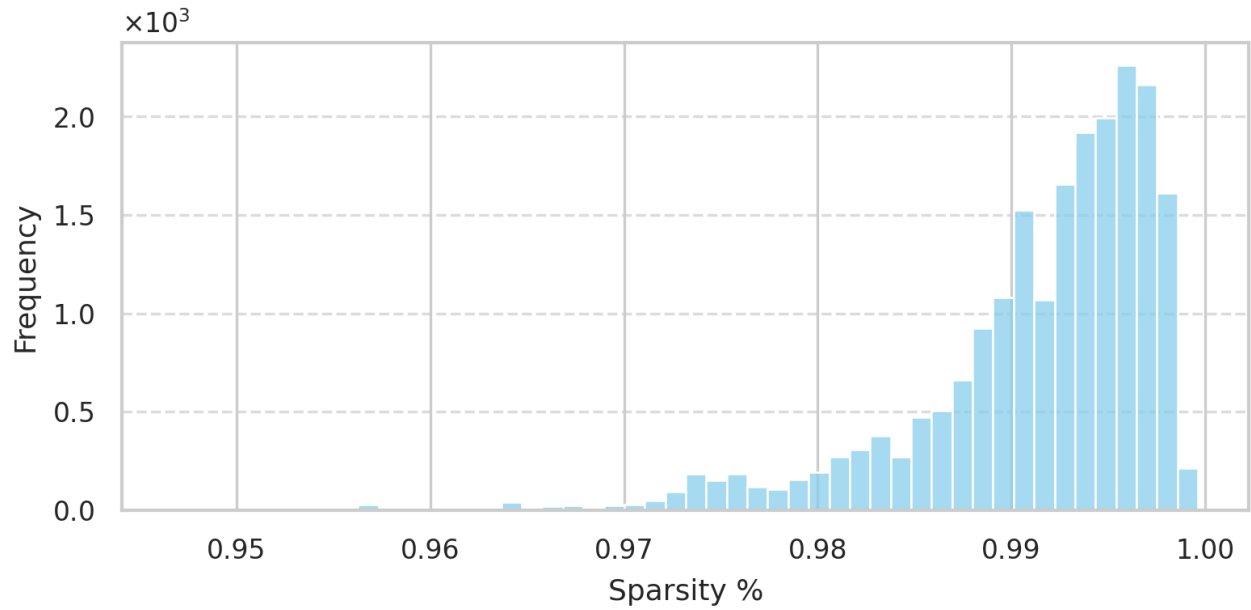


Figure C.1: Histogram showing sparsity of all matrices computed while reducing the `cksum` trace.

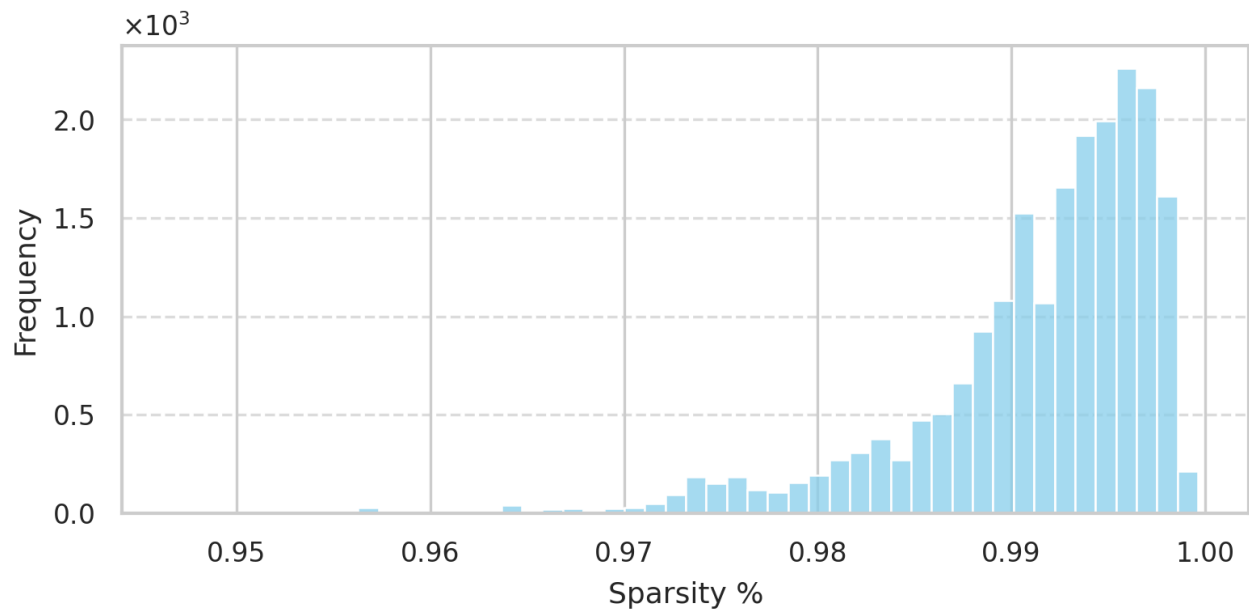


Figure C.2: Histogram showing sparsity of all matrices computed while reducing the `cksum32` trace.

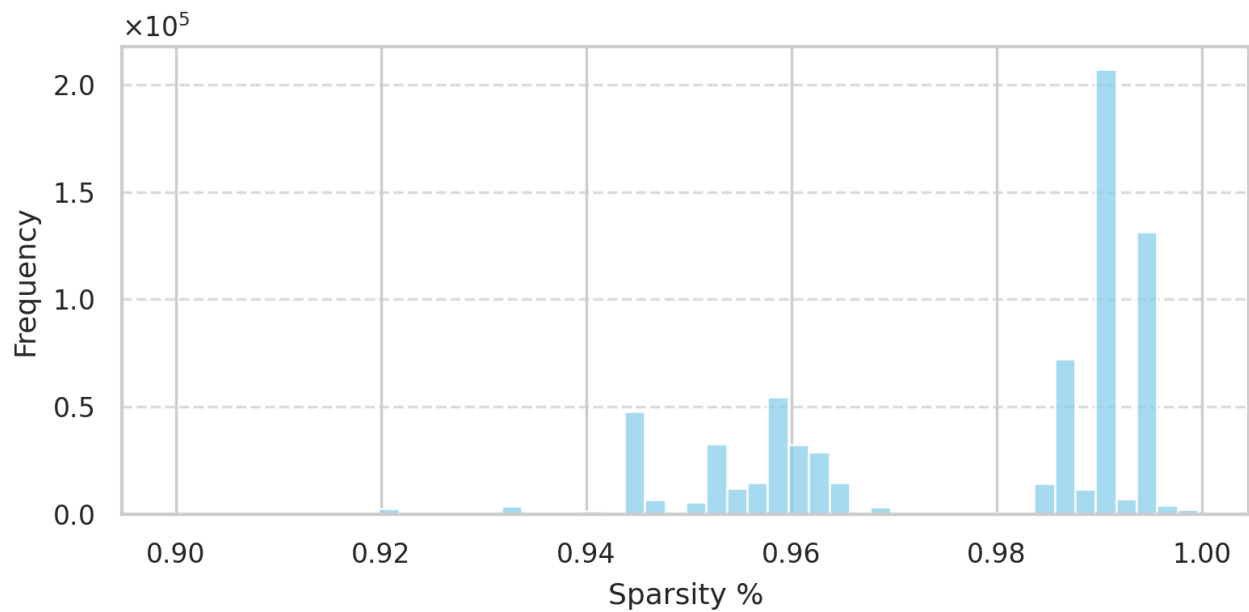


Figure C.3: Histogram showing sparsity of all matrices computed while reducing the `factor` trace.

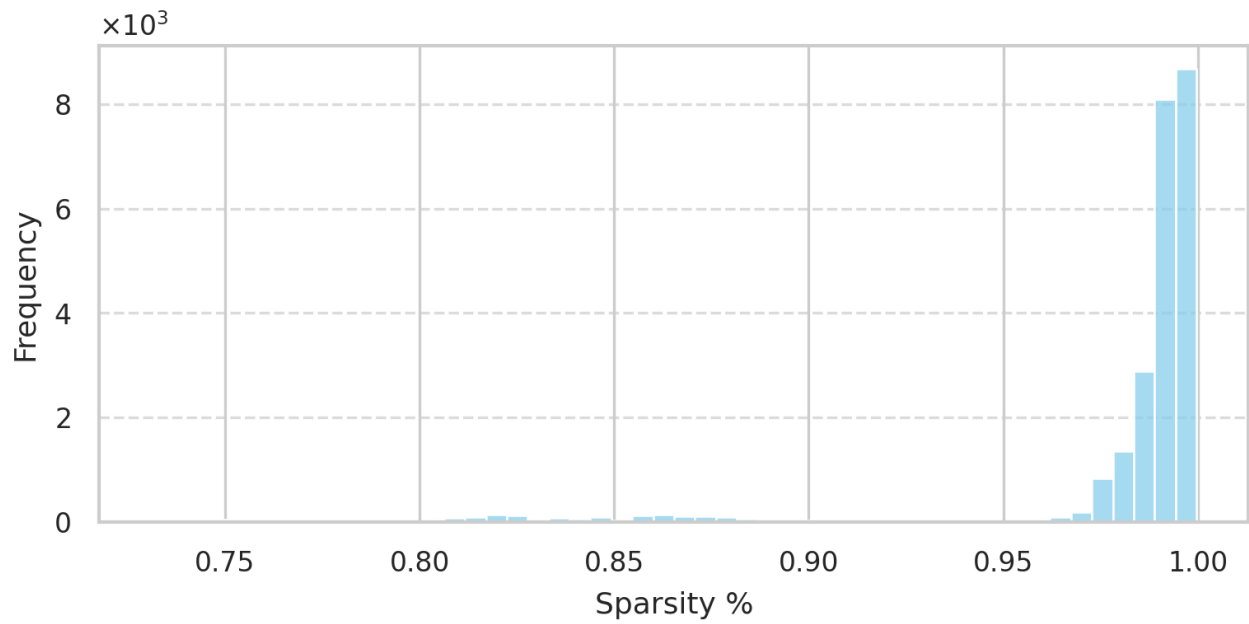


Figure C.4: Histogram showing sparsity of all matrices computed while reducing the `sha1sum` trace.

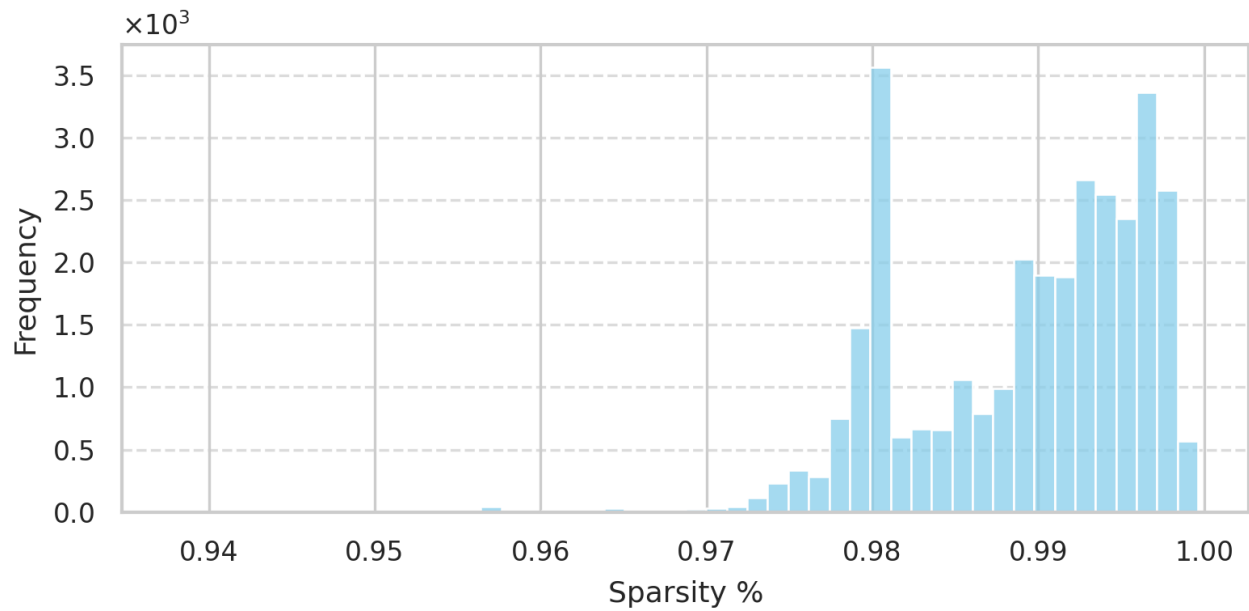


Figure C.5: Histogram showing sparsity of all matrices computed while reducing the `shuf` trace.

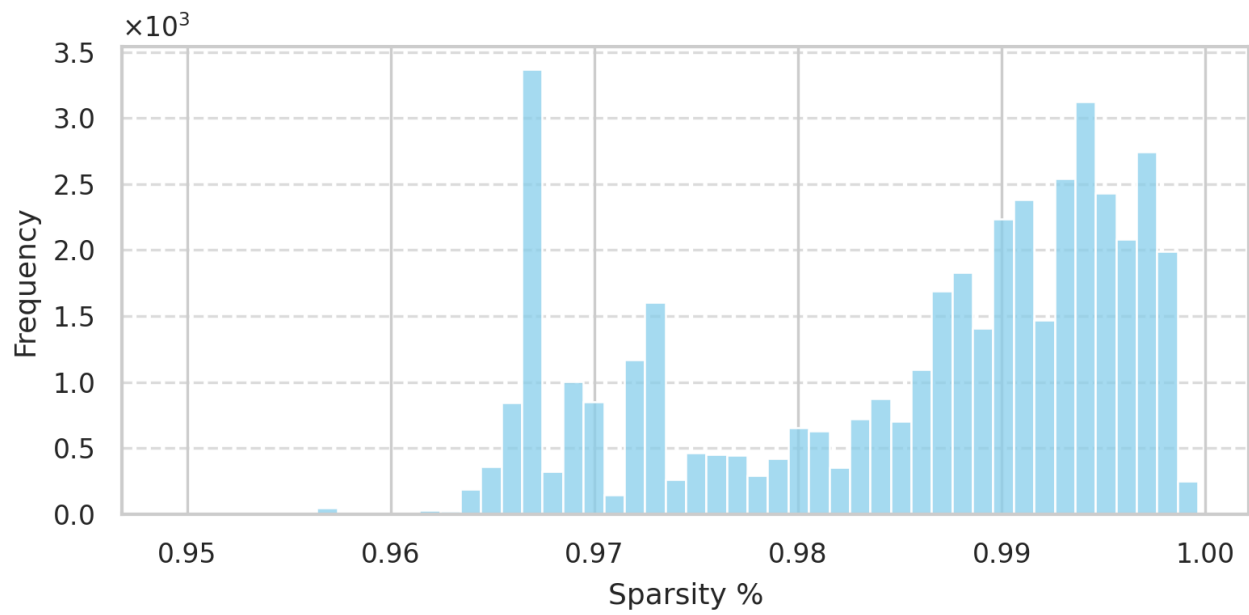


Figure C.6: Histogram showing sparsity of all matrices computed while reducing the `sort` trace.

Project Proposal

Efficient instruction-level parallelism analysis using tropical semiring ‘matrix multiplication’.

3450C

October 10, 2025

Project supervisors are listed below:

D2D supervisor Alexandra Chadwick, *awc32*

Marking supervisor RD Mullins, *rdm34*

1 Introduction

Instruction-level parallelism (ILP) is a technique commonly used by high-performance processors to improve performance. Previous studies [3] have investigated the theoretical limits of ILP in programs using models of their dynamic execution.

An upper bound for parallelism in a program arises from its critical path: the longest chain of dependent instructions in the program’s dynamic execution. For a machine that takes at least one cycle per instruction, the program cannot be executed faster than the time it takes to run the critical path sequentially.

Therefore an upper bound for ILP in a program can be calculated as the number of dynamic instructions in the program’s execution divided by the length of the critical path.

The dependencies of instructions in a program execution can be represented by its full dynamic dependency graph (DDG) [1], which is a directed acyclic graph with one node per dynamically executed instruction in a program. The edges of the graph represent dependencies between dynamically executed instructions, and so the length of the critical path is the length of the longest path through the program’s DDG.

Tropical analysis is a branch of pure mathematics using the tropical semiring [4], also known as the min/max-plus algebra, which is the semiring

$$(\mathbb{R} \cup \{+\infty\}, \oplus, \otimes)$$

with operations

$$a \oplus b = \min(a, b) \text{ or } \max(a, b)$$

$$a \otimes b = a + b$$

A recent suggestion by Chadwick [2] is to use matrix multiplication in the max tropical semiring to find the longest path in a DAG, and use this for the purposes of ILP analysis.

In this project I will create a library for performing such matrix calculations efficiently, considering aspects such as vectorisation, tiling, and representation sparsity. I will work with CPU-based compute and may work with GPU-based compute as an extension.

This introduction is adapted from the abstract of Chadwick [2].

2 The project

2.1 Project core

First, a CPU-based library for performing individual matrix computations in the tropical semiring should be implemented, using a dense representation of matrices. Techniques such as vectorisation and tiling should be considered for performance gains.

Matrix multiplication expressions will be generated from a program execution's dependency structure as a graph, with one node per register per dynamic instruction, in the column-based representation proposed in Section I-B of Chadwick [2].

The aforementioned matrix computation library will be used to compute the results of these expressions, using techniques such as caching and evaluation order optimisation to maximise efficiency.

Finally, a benchmark suite for testing performance of implementations will be implemented, focusing on testing key metrics such as execution time, memory usage, and correctness, as outlined in Section 3.2. This will allow for a systematic comparison of implementations. A baseline performance will be established using a 'classical' technique for computing dependency chains (i.e. simulating one instruction at a time naively, computing its dependencies one-by-one).

2.2 Project extensions

2.2.1 Sparse matrix representation

The core of the project implements a matrix computation library using a dense representation of matrices. In an effort to improve memory usage, I will implement a similar CPU-based matrix computation library using a sparse representation of matrices as an extension to the project core. This

may potentially improve performance too, as the cost of sparse multiplication is proportional to the number of non-zeroes in the matrix, rather than just the number of rows and columns.

The sparse library will be benchmarked against the dense library for register-only program execution dependency graphs.

2.2.2 GPU compute

The core of the project implements a CPU-based matrix computation library. To exploit the excellent parallel power of GPUs, I will implement two GPU-based matrix computation libraries, for dense and sparse representations of matrices, using a programming model such as OpenCL or CUDA.

The GPU libraries will be benchmarked against their CPU counterparts for register-only program execution dependency graphs.

2.2.3 Through-memory dependencies

So far we have only computed results of matrix expressions generated from register-only program execution dependency graphs. However, for real programs, the true critical path often involves memory operations.

A simple approach is to naively include memory locations as graph nodes alongside registers for each dynamic instruction, but this results in a catastrophic explosion of matrix sizes. Nevertheless, these matrices are extremely sparse, as most memory operations access only two or three locations at a time, and so are usable with sparse representations. For this approach, I will benchmark the performance of the two previously implemented sparse matrix computation libraries (running on CPU and GPU) against the baseline.

An alternative approach is to model the effects of memory operations using tropical semiring *addition* rather than just multiplication, as described in Section II-B of Chadwick [2]. For this approach, I will benchmark the performance of the two previously implemented dense matrix computation libraries (running on CPU and GPU) against the baseline.

3 Starting point

Existing standard libraries for matrix multiplication cannot be used as they are specialised for linear algebra (as opposed to tropical algebra). Additionally, techniques such as Strassen’s algorithm do not apply in a semiring.

A suite of tools will be provided by Alexandra Chadwick, the project supervisor, to generate matrix expressions from program executions. The

tools will do the following:

- Compute the static dependencies of every static instruction in a program.
- Turn the above into a matrix per instruction.
- Dump the dynamic flow of execution of a program.
- Use the dynamic flow and the per-instruction matrices to create a matrix expression.

Generated matrix expressions will be used as input to the matrix computation libraries. I have not started any work on the project implementation.

4 Success criteria and evaluation plan

4.1 Success criteria

For the dissertation to be considered a success, all items in the project core (Section 2.1) should be completed, specifically the following:

- Implement a CPU-based library for performing individual matrix computations in the tropical semiring, using a dense representations of matrices.
- Use the CPU-based library to compute the result of large matrix expressions for computing longest paths in register-only DDGs.
- Establish a baseline performance using a classical technique for computing dependency chains.
- Implement a benchmark suite for testing performance of implementations.

For the core to be considered a success, the CPU-based library should demonstrate a performance improvement over the naive baseline for computing the length of the critical path.

If time permits, I will complete the three project extensions described in Section 2.2, prioritising them in the order they are presented. The success criteria for each extension is as follows:

- Sparse matrix representation: The implementation from this extension should demonstrate a reduction in memory usage and an improvement in performance for very sparse matrices over the CPU-based dense implementation from the project core.

- GPU compute: The implementations from this extension should show performance improvement over their CPU-based counterparts.
- Through-memory dependencies: The benchmarks from this extension should demonstrate a performance improvement for the matrix computation libraries over the naive baseline for computing the length of the critical path in a full DDG.

All implementations of the matrix computation library should evaluate matrix expressions **correctly**.

4.2 Evaluation plan

The evaluation of the matrix computation libraries will be conducted using a comprehensive set of benchmarks on both register-only DDGs and ones with through-memory dependencies (addition and sparse representations). All implementations will be benchmarked against the naive baseline (sequential dependency chain computation) to quantify performance improvements.

For each benchmark, the following metrics will be collected:

Execution time: Total time taken to compute the longest path, or evaluate the matrix expression.

Memory usage: Peak and average memory consumption during computation.

Correctness: Verification that computed results match expected results for known cases.

Results will be analysed to determine which approach is most effective for each situation, and to validate the success criteria detailed above.

5 Plan of work

10/10-24/10: Implement naive baseline

Doing this before working with matrix computations will allow me to understand the starting state of dependency chain computation.

Deliverable: Naive baseline.

25/10-7/11: Implement CPU-based dense matrix computation library

I will first implement the library for individual matrix multiplications, focusing on performance optimisations.

Deliverable: Individual matrix computation library implemented.

8/11-21/11: Use CPU-based library to compute matrix expressions

I will use the aforementioned library in computing the results of large matrix expressions. Work from this sprint may spill over into the next sprint due to an assessment deadline for the Category Theory module I am taking.

Deliverable: Library for computing matrix expressions implemented.

22/11-5/12: Catch up and benchmark suite

First I will complete any outstanding work from the previous sprint, and develop a benchmark suite for testing solution performance in preparation for evaluation.

Deliverable: Benchmark suite implemented.

6/12-19/12: Evaluation of core

I will evaluate the results of the project core against the success criteria using the benchmark suite developed in the previous sprint.

Deliverable: Evaluation of core completed.

20/12-2/1: Low-progress buffer

As this sprint falls over the holiday season, the amount of work done may be minimal. I will use this time as a buffer for any outstanding work to be completed.

3/1-16/1: Sparse matrix representation

This is the first of the project extensions.

Deliverable: Matrix computation library using sparse representation implemented.

17/1-30/1: GPU-based matrix computation libraries

This is the second of the project extensions. Due to an assessment deadline for the Category Theory module I am taking over the first week of Lent term, work from this sprint may spill over into the next sprint.

Deliverable: Matrix computation library using GPU compute implemented.

31/1-13/2: Catch up and progress report

I will catch up on any outstanding work, and dedicate time to preparing the progress report and presentation due on 06/02/2025.

14/2-27/2: Evaluation of first two extensions

I will evaluate the results of the first two project extensions against the success criteria.

Deliverable: Evaluation of first two extensions completed.

28/2-13/3: Investigate through-memory dependencies

The second half of this sprint will coincide with an assessment deadline for the Multicore Semantics and Programming module I am taking, and so progress may be limited.

This is the third of the project extensions. This extension is an investigation of the performance of the previously implemented libraries on full DDGs, and so may not require as much work as the previous extensions.

Deliverable: Investigation of through-memory dependencies completed.

14/3-27/3: Catch up and evaluate last extension

The first half of this sprint will coincide with an assessment deadline for the Multicore Semantics and Programming module I am taking, and so progress may be limited.

This sprint will be used to catch up on any outstanding work, and complete evaluations for all extensions.

Deliverable: Evaluation of all extensions completed.

28/3-10/4: Start write-up

In this sprint I will plan the dissertation structure, and start writing chapters 1 (introduction) and 2 (preparation) of the dissertation for review.

If time permits, I will start writing chapter 3 (implementation).

Deliverable: Dissertation document plan drafted, and chapters 1 and 2 written.

11/4-24/4: Continue write-up

I will address feedback received on chapters already written, then I will continue writing the dissertation, aiming to complete chapters 3 (implementation), 4 (evaluation), and 5 (conclusions).

Deliverable: Dissertation chapters 3, 4 and 5 written.

25/4-8/5: Submit dissertation

I will address feedback on all the writing done in previous sprints, and then proofread the dissertation to be ready for submission 1 week before the deadline of 15/05/2025.

6 Resource declaration

I will use my personal laptop (Intel i7-12700H, 16GB RAM, 1TB SSD) for developing the code artefact and writing the thesis. I will use the Git VCS for both the code and thesis source, as two separate repositories hosted on Github using a personal account. I will make an effort to push any changes to the remote repository as soon as they are committed, at least once a day.

In the event of my device failing, I will use university-provided machines in my college's computer room or library until I am able to acquire a new laptop.

I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure.

References

- [1] T. AUSTIN AND G. SOHI, *Dynamic dependency analysis of ordinary programs*, in [1992] Proceedings the 19th Annual International Symposium on Computer Architecture, 1992, pp. 342–351.
- [2] A. CHADWICK, *On tropical semirings and the limits of instruction-level parallelism*. <https://www.cl.cam.ac.uk/~awc32/tropical-ilp.pdf>, 2025.
- [3] A. W. CHADWICK, M. ERDŐS, U. BORA, A. BHOSALE, B. LYTTON, Y. GUO, R. COOPER, G. GABRIELLI, AND T. M. JONES, *The future*

of instruction-level parallelism (ILP), in 2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2025, pp. 350–352.

- [4] J.-E. PIN, *Tropical Semirings*, in Idempotency (Bristol, 1994), J. Gunawardena, ed., Publ. Newton Inst. 11, Cambridge Univ. Press, Cambridge, 1998, pp. 50–69.